

ACTIVITY GUIDE FOR TERRAFORM

TABLE OF CONTENTS

1. Practical Lesson 1 : Understanding infrastructure as code (IaC) concepts	3
No Practical for this lesson.....	3
2. Practical Lesson 2: Installing terraform.....	4
Refer to environment setup file for installation	4
3. Practical Lesson 3: Introduction To Terraform	5
Practical lesson 3.1: Configuring an Oracle Cloud Infrastructure (OCI) Provider	5
Practical Lesson 3.2: Oracle Cloud Infrastructure (OCI) General Information	9
4. Practical Lesson 4 : Terraform Basics	12
Practical Lesson 4.1: Creating OCI compartments	12
Practical Lesson 4.2 Oracle Cloud Infrastructure (Oci) Compute Instance	15
5. Practical Lesson 5: Use Terraform outside of core workflow	19
PRACTICAL LESSON 5.1 : USING IMPORT FOR IMPORT RESOURCES.....	19
PRACTICAL LESSON 5.2: CREATING A VIRTUAL CLOUD NETWORK.....	22
6. Practical Lesson 6 : Interact With Modules	26
PRACTICAL LESSON 6.1: SET UP A SIMPLE INFRASTRUCTURE WITH OCI TERRAFORM	26
7. Practical Lesson 7 : Use the core Terraform workflow	31
PRACTICAL LESSON 7.1 : CREATE A KUBERNETES CLUSTER	31
8. Practical Lesson 8: Implement And Maintain State	42
PRACTICAL LESSON 8.1: Oracle Cloud Infrastructure (OCI) Autonomous Database Warehouse (ADW).....	42
9. Practical Lesson 9: Read, generate and modify configuration	45
Practical Lesson 9.1: CREATING AN OBJECT STORAGE BUCKET ON OCI	45
10. Practical Lesson 10: Understand HCP Terraform Cloud Capabilities.....	47
Practical Lesson 10.1: Sign Up Terraform cloud.....	47
Practical Lesson 10.2:: CLI Log In.....	49
Practical Lesson 10.3: Creating Credential Variable Set.....	51
Practical Lesson 10.4: Creating a workspace.....	55
Practical Lesson 10.5:Creating an infrastructure on AWS	59

1. Practical Lesson 1 : Understanding infrastructure as code (IaC) concepts

No Practical for this lesson.

2. Practical Lesson 2: Installing terraform

Refer to environment setup file for installation

3. Practical Lesson 3: Introduction To Terraform

Practical lesson 3.1: Configuring an Oracle Cloud Infrastructure (OCI) Provider

Overview: The Oracle Cloud Infrastructure (OCI) provider allows you to use Terraform to interact with Oracle Cloud Infrastructure resources.

Prerequisites:

-
- Oracle Cloud infrastructure Account
 - Access to Cloud-shell
 - An internet connection

Tasks:

- Add Public Key to Oracle Cloud Account
- Gather Information From the Oracle Cloud Account
- Create Working Directory
- oci_provider.tf
- oci_provider_variables.auto.tfvars
- Test the OCI Provider

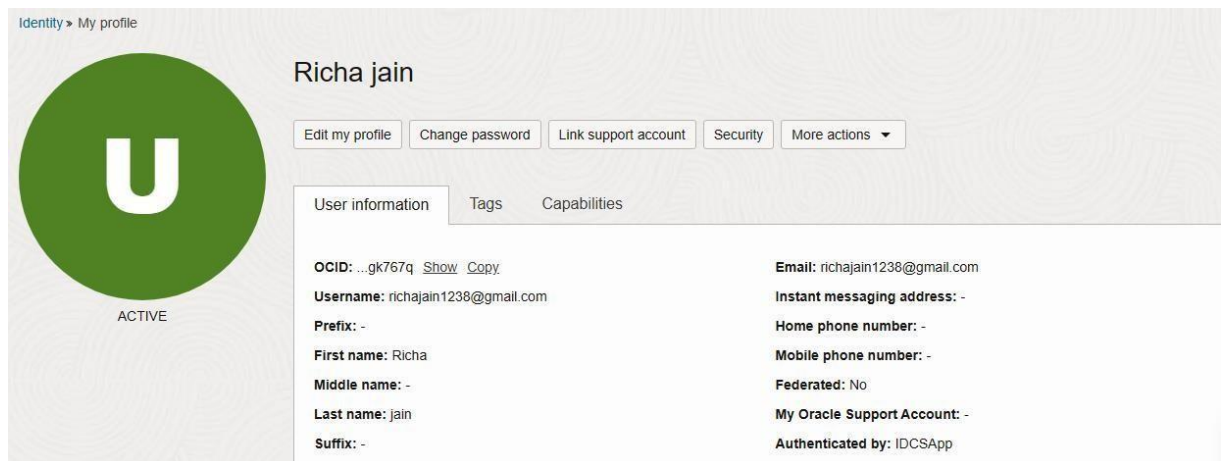
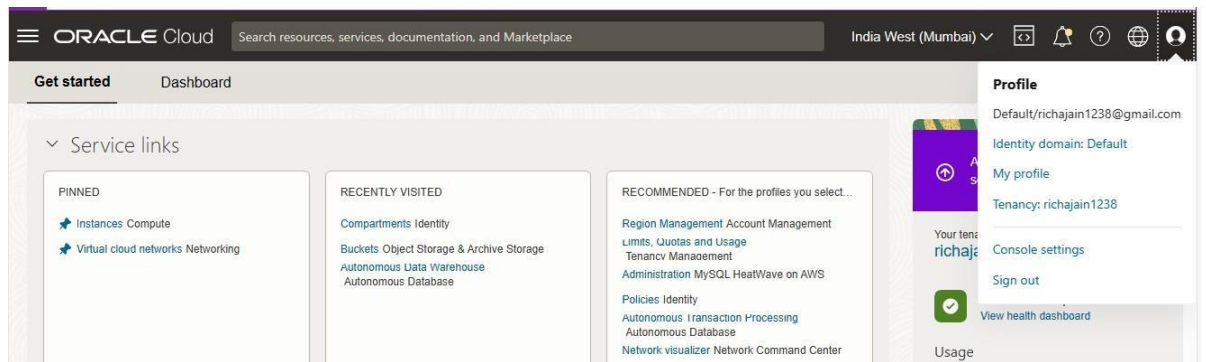
See the environment setup file for generating the SSH keys, public keys , API keys and Fingerprints and other information from the OCI cloud account.

Generating the Public key

Add Public Key to Oracle Cloud Account

Generate a key pair as described [here](#). You will need to upload the public key to your Oracle Cloud account to allow Terraform to authenticate to your Oracle Cloud account.

- Log in to your Oracle Cloud account.
- From the Hamburger menu at the top-left of the screen, select the "Identity > Users" option (in case you have many users). If there is only one user, go to top right "Profile" option and select my profile. This is the user that will run the Terraform commands.



From here, copy the OCID (Save this on notepad for future reference)

- Scroll down on the same page and under the resources section, Click on the "API Keys" link in the "Resources" menu on the left-bottom of the screen.
- Click the "Add API Key" button.
- Select the "PASTE PUBLIC KEY" option.
- Paste in your public key.
- Click the "Add" button.
- Make a note of the Fingerprint associated with the API Key. It will be used later.

Note: You can also refer the Environment setup file for the Public key generation and its usage

Use the inbuilt cloud shell's code editor to create the terraform files

Configuring the OCI provider

1. Create a directory oci_provider : **mkdir oci_provider**
2. Move to that directory : **cd oci_provider**
3. Create a file with .tf extension ,i.e., **Oci_provider.tf**

```

1  # Variables.
2  variable "tenancy_ocid"      { type = string }
3  variable "user_ocid"        { type = string }
4  variable "private_key_path" { type = string }
5  variable "fingerprint"      { type = string }
6  variable "region"           { type = string }
7  variable "root_compartment_id" { type = string }
8
9
10 # Resources
11 provider "oci" {
12   tenancy_ocid      = var.tenancy_ocid
13   user_ocid         = var.user_ocid
14   private_key_path  = var.private_key_path
15   fingerprint       = var.fingerprint
16   region            = var.region
17 }

```

The provider block is specifying the necessary configuration to connect to the Oracle Cloud Infrastructure, including:

- **tenancy_ocid** : The OCID (Oracle Cloud Identifier) for your tenancy.
- **user_ocid**: The OCID for the user accessing the Oracle Cloud services.
- **private_key_path**: The path to the private key file used for authentication.
- **fingerprint**: The fingerprint of the public key associated with the private key.
- **region**: The Oracle Cloud Infrastructure region.
- **root_compartment_id**: The OCID of the root compartment in Oracle Cloud.

The file begins with variable definitions. We could set default values for these variables, or use literal values directly in the provider definition, but we don't want sensitive information checked into version control, so it makes sense to separate out variable values from the script. The resources section defines the provider using the input variables.

The variables and resources sections can be split into separate files if you find that organisation easier. It may help for more complex definitions.

You'll notice the `root_container_id` variable isn't used in the script. We could just remove it altogether, but it is used by many of the OCI resources, so we've included it in our provider variable file below. If we don't include it here we will get a warning.

4. Create Another file : `oci_provider_variables.auto.tfvars`

Create a file called "`oci_provider_variables.auto.tfvars`" with the following contents. Adjust the values to match the information you gathered from the Oracle Cloud account.

```

1  tenancy_ocid      = "ocid1.tenancy.oc1..aaaaaaaadbra67yxbxg7uh55ewitjdjncmfy5bzyzfbh2f2v74wbgfmwstsa"
2  user_ocid         = "ocid1.user.oc1..aaaaaaaaym63hn6qkocbclb5omkmvoun3dhos4q2b2gvagj6k4bhrygk767q"
3  private_key_path  = "~/.apikey/oci_api_key_public.pem"
4  fingerprint       = "21:f4:3f:a0:2a:d1:1d:44:f3:fa:ef:0c:eb:fa:5a:1e"
5  region            = "ap-mumbai-1"
6  root_compartment_id = "ocid1.tenancy.oc1..aaaaaaaadbra67yxbxg7uh55ewitjdjncmfy5bzyzfbh2f2v74wbgfmwstsa"

```

5. Test the OCI Provider

Initialize the working directory using the terraform init command.

terraform init

```

richajain1@cloudshell:oci_provider (ap-mumbai-1)$ terraform init

Initializing the backend...

Initializing provider plugins...
- Reusing previous version of hashicorp/oci from the dependency lock file
- Using previously-installed hashicorp/oci v5.18.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see
any changes that are required for your infrastructure. All Terraform commands
should now work.

If you ever set or change modules or backend configuration for Terraform,
rerun this command to reinitialize your working directory. If you forget, other
commands will detect it and remind you to do so if necessary.
richajain1@cloudshell:oci_provider (ap-mumbai-1)$

```

6. terraform plan

```

richajain1@cloudshell:oci_provider (ap-mumbai-1)$ terraform plan

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.
richajain1@cloudshell:oci_provider (ap-mumbai-1)$

```

7. terraform apply

```

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.
richajain1@cloudshell:oci_provider (ap-mumbai-1)$ terraform apply

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.
richajain1@cloudshell:oci_provider (ap-mumbai-1)$

```


Practical Lesson 3.2: Oracle Cloud Infrastructure (OCI) General Information

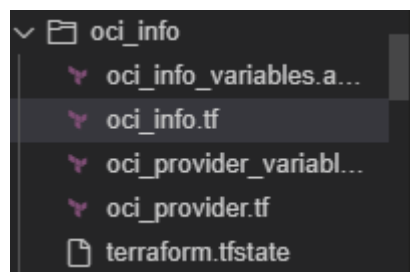
Objective: describes how to display some commonly used information from Oracle Cloud Infrastructure (OCI), needed when using Terraform. This script is displaying information, not creating any new resources.

Prerequisites :

1. A paid Oracle Cloud Infrastructure account. See [Signing Up for Oracle Cloud Infrastructure](#).
2. A MacOS, Linux, or Windows computer.

Tasks

Note: Make sure that you have all the .tf files as given in the snapshot



1. Create a directory oci_provider : **mkdir oci_info**
2. Move to that directory : **cd oci_info**
3. create the Oci_provider.tf and **oci_provider_variables.auto.tfvars** into this directory as these are required to configure the resources on oci
4. Create a file, "**Oci_info.tf**" with following code

```
1. # Variables
2. variable "compartment_id" { type = string }
3.
4. variable "operating_system" {
5.   type    = string
6.   default = "Oracle Linux"
7. }
8.
9. variable "operating_system_version" {
10.  type    = string
11.  default = "8"
12.}
13.
14. variable "shape" {
15.   type    = string
16.   default = "VM.Standard.E2.1.Micro"
17.}
```

```

18.
19. # Resources
20. data "oci_identity_availability_domains" "ads" {
21.   compartment_id = var.compartment_id
22. }
23.
24. data "oci_core_images" "d_images" {
25.   compartment_id      = var.compartment_id
26.   operating_system    = var.operating_system
27.   operating_system_version = var.operating_system_version
28.   shape               = var.shape
29. }
30.
31. data "oci_core_shapes" "d_shapes" {
32.   compartment_id      = var.compartment_id
33.   image_id            = data.oci_core_images.d_images.images[0].id
34. }
35.
36. # Outputs
37. output "first_availability_domain_name" {
38.   value = data.oci_identity_availability_domains.ads.availability_domains
39.     [0].name
40. }
41. output "availability_domain_names" {
42.   value = data.oci_identity_availability_domains.ads.availability_domains
43.     [*].name
44. }
45. output "latest_image_id" {
46.   value = data.oci_core_images.d_images.images[0].id
47. }
48.
49. output "shape_names" {
50.   value = data.oci_core_shapes.d_shapes.shapes[*].name
51. }

```

The file begins with variable definitions. We could set default values for these variables, or use literal values directly in the provider definition, but we don't want sensitive information checked into version control, so it makes sense to separate out variable values from the script. Many of the parameters are defaulted. The resources section defines availability domains, images and shapes using the input variables. The outputs section allows us to see information about the availability domains, images and shapes.

5. Then create file **"oci_info_variables.auto.tfvars"** with following required code

```

compartment_id = "ocid1.compartment.oc1..aaaaaaaac43izpi4eeaup3jielykgyqxsevnv
zvocv7m2o4re4lvhwd2et7q"
operating_system      = "Oracle Linux"
operating_system_version = "8"
shape                 = "VM.Standard.E2.1.Micro"

```

6. Run the command **terraform init** (for initialising the working directory)

7. terraform plan

```

richajain1@cloudshell:oci_info (ap-mumbai-1)$ terraform plan
data.oci_identity_availability_domains.ads: Reading...
data.oci_core_images.d_images: Reading...
data.oci_identity_availability_domains.ads: Read complete after 0s [id=IdentityAvailabilityDomainsDataSource-3574321520]
data.oci_core_images.d_images: Read complete after 0s [id=CoreImagesDataSource-1107955431]
data.oci_core_shapes.d_shapes: Reading...
data.oci_core_shapes.d_shapes: Read complete after 0s [id=CoreShapesDataSource-3070724189]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no differences, so no changes are needed.
richajain1@cloudshell:oci_info (ap-mumbai-1)$ █

```

7. terraform apply

```

Outputs:

availability_domain_names = tolist([
  "FuAZ:AP-MUMBAI-1-AD-1",
])
first_availability_domain_name = "FuAZ:AP-MUMBAI-1-AD-1"
latest_image_id = "ocid1.image.oc1.ap-mumbai-1.aaaaaaaayjh7v5x7cwns4kuldagah5pmzqffulrr4olo2e72roansqatf2xa"
shape_names = tolist([
  "VM.Standard.E2.1.Micro",
])
richajain1@cloudshell:oci_info (ap-mumbai-1)$ █

```

This activity describes how to display some commonly used information from Oracle Cloud Infrastructure (OCI), needed when using terraform. This script is displaying information, not creating any new resources.

4. Practical Lesson 4 : Terraform Basics

Overview: Here you will learn how to use the terraform to deploy resources

Practical Lesson 4.1: Creating OCI compartments

Overview:

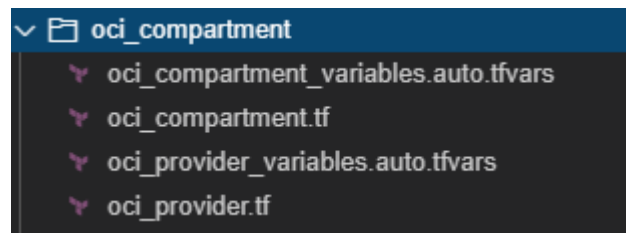
A compartment is a logical container, to organize and control access to the **Oracle Cloud Infrastructure (OCI) Resources** (*Compute, Storage, Network, Load Balancer, etc*) created within that compartment and you impose some policies to that compartment, which restricts **who can use the resources** created within than compartment other than administrators of your account.

Prerequisites:

- Oracle Cloud infrastructure Account
- An internet connection

Tasks

Make sure to include the below .tf files to your working directory



1. Create a directory : **mkdir oci_compartment**
2. Move to that directory : **cd oci_compartment**
3. create the **Oci_provider.tf** and **oci_provider_variables.auto.tfvars** into this directory as these are required to configure the resources on oci
4. Create a file, "**Oci_compartment.tf**" with following code to create the compartment

```

# Variables
variable "compartment_name" { type = string }
variable "compartment_desc" { type = string }

# Resources
resource "oci_identity_compartment" "tf_compartment" {
  # Required
  compartment_id = var.root_compartment_id
  description    = var.compartment_desc
  name           = var.compartment_name
}

# Outputs
output "compartment_name" {
  value = oci_identity_compartment.tf_compartment.name
}

output "compartment_id" {
  value = oci_identity_compartment.tf_compartment.id
}

```

The file begins with variable definitions. We could set default values for these variables, or use literal values directly in the provider definition, but we don't want sensitive information checked into version control, so it makes sense to separate out variable values from the script. The resources section defines the compartment using the input variables. The outputs section allows us to see information about the compartment that's been created, including the ID of the compartment.

The variables, resources and outputs sections can be split into separate files if you find that organisation easier. It may help for more complex definitions.

The root_compartment_id parameter is not defined here, as it's already defined in the oci_provider.tf file.

5. create another file : **oci_compartment_variables.auto.tfvars**

```

1 compartment_name = "comp1"
2 compartment_desc = "creating first compartment"

```

You can give any description according to your choice.

6. Initialize the working directory using **the terraform init** command.

```
terraform init
```

```
richajain1@cloudshell:oci_compartment (ap-mumbai-1)$ terraform init
```

Initializing the backend...

Initializing provider plugins...

- Reusing previous version of hashicorp/oci from the dependency lock file
- Using previously-installed hashicorp/oci v5.18.0

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.

If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.

```
richajain1@cloudshell:oci_compartment (ap-mumbai-1)$
```

7. Use the terraform plan command to test the execution plan.
terraform plan

8. Use the terraform apply command to create the OCI compartment.

terraform apply

Identity

Overview

Domains

Network Sources

Policies

Compartments

Filters

State

Active | Deleting

Tag filters

add | clear

no tag filters applied

Compartments

Create Compartment

Name	Status	OCID	Authorized	Security Zone ⓘ	Subcompartments	Created
richajain1238	● Active	...mws1s a	Yes	-	4	-
comp1	● Active	...jod1fa	Yes	-	0	Thu, Nov 2, 2023, 10:59:2 5 UTC
ManagedCompartmentF orPaaS	● Active	...d2et7g	Yes	-	0	Sat, Sep 2, 2023, 02:10:30 UTC
Richajain	● Active	...qjsvpa	Yes	-	0	Wed, Aug 30, 2023, 20:56: 32 UTC
Richajain12	● Active	...aplh3g	Yes	-	0	Wed, Aug 30, 2023, 21:01: 56 UTC
						Showing 5 items < 1 of 1 >

Practical Lesson 4 .2 Oracle Cloud Infrastructure (Oci) Compute Instance

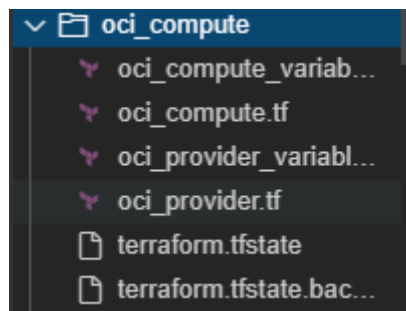
Overview : Oracle Cloud Infrastructure Compute lets you provision and manage compute hosts, known as instances . You can create instances as needed to meet your compute and application requirements.

Prerequisites:

- Oracle Cloud infrastructure Account
- Access to Cloud-shell
- An internet connection

Tasks:

Note : include all the .tf files as mentioned in the snapshot



1. Create a working directory and **mkdir oci_compute**
2. Move to that directory **Cd oci_compute**
3. Create the same **oci_provider.tf** and **oci_provider.auto.tfvars** file (created earlier)
4. Create a file called "**oci_compute.tf**" with the following contents.

```
# Variables
variable "compartment_id"           { type = string }
variable "compute_name"             { type = string }
variable "compute_subnet_id"        { type = string }
variable "compute_image_id"         { type = string }
variable "compute_ssh_authorized_keys" { type = string }

variable "compute_shape" {
  type    = string
  default = "VM.Standard.E2.1.Micro"
}

variable "compute_cpus" {
  type    = string
  default = "1"
}
```

```

variable "compute_memory_in_gbs" {
  type    = string
  default = "1"
}

# Resources
data "oci_identity_availability_domains" "ads" {
  compartment_id = var.compartment_id
}

resource "oci_core_instance" "tf_compute" {
  # Required
  availability_domain = data.oci_identity_availability_domains.ads.availabilit
y_domains[0].name
  compartment_id      = var.compartment_id
  shape               = var.compute_shape

  source_details {
    source_id      = var.compute_image_id
    source_type    = "image"
  }

  # Optional
  display_name      = var.compute_name

  shape_config {
    ocpus          = var.compute_cpus
    memory_in_gbs = var.compute_memory_in_gbs
  }

  create_vnic_details {
    subnet_id      = var.compute_subnet_id
    assign_public_ip = true
  }

  metadata = {
    ssh_authorized_keys = file(var.compute_ssh_authorized_keys)
  }

  preserve_boot_volume = false
}

# Outputs
output "compute_id" {
  value = oci_core_instance.tf_compute.id
}

```



```

output "db_state" {
  value = oci_core_instance.tf_compute.state
}

output "compute_public_ip" {
  value = oci_core_instance.tf_compute.public_ip
}

```

The file begins with variable definitions. We could set default values for these variables, or use literal values directly in the provider definition, but we don't want sensitive information checked into version control, so it makes sense to separate out variable values from the script. Many of the parameters are defaulted. The resources section defines the compute instance using the input variables. The outputs section allows us to see information about the compute instance that's been created, including the name and state.

5. **Create `oci_compute_variables.auto.tfvars`** (to provide the values to the variables related to the creating the compute instance)

```

compartment_id      = "ocid1.compartment.oc1..aaaaaaaep1rbcpgxopdimbh
5befcepyfhd62fb44iifehqelgk3jnjodrla"
compute_shape       = "VM.Standard.E2.1.Micro"
compute_name        = "myvm1"
compute_subnet_id   = "ocid1.subnet.oc1.ap-mumbai-
1.aaaaaaaad2ceplw4jd53mibcwnsw2vbus3qwaj4y4ynsnsz64swiczvs6yyq"
compute_image_id    = "ocid1.image.oc1.ap-mumbai-
1.aaaaaaaacd4r5h6lh4omldajhnizvqwpjhtnphoauode2nzvsswq5aa2gvq"
compute_ssh_authorized_keys = "/home/richajain1/oci.pub"

```

The `compartment_id` is the OCID of the compartment that will house the compute instance. You must adjust it with a valid value from your Oracle Cloud account. You would not normally use the root compartment for this. You can get the ID of a compartment from your Oracle Cloud account as follows.

- Top-Left Hamburger > Identity > Compartments
- Click on the compartment of interest.
- Click the "Copy" link next to "OCID".

The `db_subnet_id` is the OCID of the subnet the compute instance will be connected to.

- Top-Left Hamburger > Networking > Virtual Cloud Networks
- Click on the VCN of interest.

- Click on the kebab menu to the far right of the subnet of interest.
- Select the "Copy OCID" option on the resulting popup menu.

The `compute_image_id` is the OCID of the OS image the compute instance will be based upon.

Note : To see the list of compute image id for Oracle Linux 8 for the particular compartment

...

```
oci compute image list --compartment-id <your-compartment-ocid> --operating-system
"Oracle Linux" --operating-system-version "8"
```

6. terraform init
7. terraform plan
8. terraform apply

This will create the compute instance in the mentioned compartment in your tfvars file. To check the newly built compartment, Go to hamburger menu and select compute instance. Choose the compartment. It will display the compute instance

Instances *in comp1 compartment*

An [instance](#) is a compute host. Choose between virtual machines (VMs) and bare metal instances. The image that you use to launch an instance determines its operating system and other software.

Create instance	Actions ▼							
☐	Name	State	Public IP	Private IP	Shape	OCPU count	Memory (GB)	Availability
☐	myvm1 Always Free	Running	129.154.252.197	10.0.0.3	VM.Standard.E2.1.Micro	1	1	AD-1
☐	myvm1 Always Free	Running	141.148.221.239	10.0.0.145	VM.Standard.E2.1.Micro	1	1	AD-1

0 selected

Showing 2 items < 1 of 1 >

5. Practical Lesson 5: Use Terraform outside of core workflow

PRACTICAL LESSON 5.1 : USING IMPORT FOR IMPORT RESOURCES

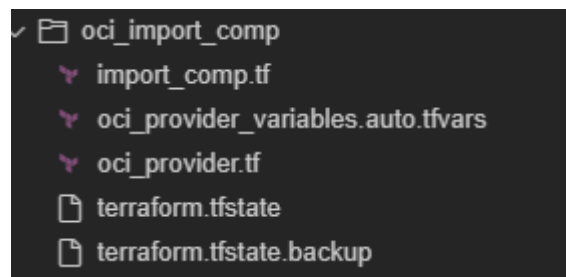
Overview : The terraform import command is used in Terraform to associate an existing resource in your cloud provider environment with a Terraform configuration. Prerequisites:

Prerequisites:

- Oracle Cloud infrastructure Account
- Access to Cloud-shell
- An internet connection

Tasks:

Note : include all the .tf files as mentioned in the snapshot



1. Create a working directory mkdir oci_import_comp
2. Move to that directory Cd oci_import_comp
3. Create the same oci_provider.tf and oci_provider.auto.tfvars file (created earlier)
4. Create a file named import_comp.tf with following

```
oci_import_comp > ❏ import_comp.tf
1 resource "oci_identity_compartment" "test" {
2   #Required
3   compartment_id = "ocid1.compartment.oc1..aaaaaaaaplrbcpqxdimbh5befcepyfhd62fb44iifehqelgk3jnjodrla"
4   description = "creating first compartment"
5   name = "test1"
6 }
7 output "compartment" {
8   value = oci_identity_compartment.test
9 }
```

5. Then, Import a compartment into Terraform by following command on command line

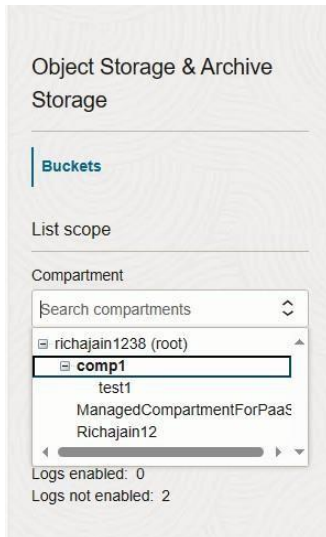
```
terraform import oci_identity_compartment.example YOUR_COMPARTMENT_OCID
```

```
richajain@cloudshell:oci_import_comp (ap-mumbai-1)$ terraform import oci_identity_compartment.test ocid1.compartment.oc1..aaaaaaaauqax7epejsqahhj6ax6uj6d5u7tbeyurmor75ikeo2yzc7qjsvpa
oci_identity_compartment.test: Importing from ID "ocid1.compartment.oc1..aaaaaaaauqax7epejsqahhj6ax6uj6d5u7tbeyurmor75ikeo2yzc7qjsvpa"...
oci_identity_compartment.test: Import prepared!
  Prepared oci_identity_compartment for import
oci_identity_compartment.test: Refreshing state... [id=ocid1.compartment.oc1..aaaaaaaauqax7epejsqahhj6ax6uj6d5u7tbeyurmor75ikeo2yzc7qjsvpa]

import successful!

The resources that were imported are shown above. These resources are now in
your Terraform state and will henceforth be managed by Terraform.
```

- To see this, go to object storage from the hamburger menu at the left. And locate the compartment in which the compartment is imported



- terraform plan** (to make that imported compartment active)

```
richajain@cloudshell:oci_import_comp (ap-mumbai-1)$ terraform plan
oci_identity_compartment.test: Refreshing state... [id=ocid1.compartment.oc1..aaaaaaaauqax7epejsqahhj6ax6uj6d5u7tbeyurmor75ikeo2yzc7qjsvpa]
oci_identity_compartment.test_compartment: Refreshing state... [id=ocid1.compartment.oc1..aaaaaaaauqax7epejsqahhj6ax6uj6d5u7tbeyurmor75ikeo2yzc7qjsvpa]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
~ update in-place

Terraform will perform the following actions:

# oci_identity_compartment.test will be updated in-place
~ resource "oci_identity_compartment" "test" {
  ~ compartment_id = "ocid1.tenancy.oc1..aaaaaaaadbra67yxbxg7uh55ewitjdjncmfy5bzyzfbh2f2v74wbqfwmstsa" -> "ocid1.compartment.oc1..aaaaaaaep1rbcpqxpodimbh5befcepyfhd62fb44iifehqelgk3jnjdrla"
  ~ description    = "ocicloud" -> "creating first compartment"
    id             = "ocid1.compartment.oc1..aaaaaaaauqax7epejsqahhj6ax6uj6d5u7tbeyurmor75ikeo2yzc7qjsvpa"
  ~ name          = "Richajain" -> "test1"
    # (5 unchanged attributes hidden)

  ~ timeouts {}
}

Plan: 0 to add, 1 to change, 0 to destroy.

Changes to Outputs:
~ compartment = {
  ~ compartment_id = "ocid1.tenancy.oc1..aaaaaaaadbra67yxbxg7uh55ewitjdjncmfy5bzyzfbh2f2v74wbqfwmstsa" -> "ocid1.compartment.oc1..aaaaaaaep1rbcpqxpodimbh5befcepyfhd62fb44iifehqelgk3jnjdrla"
  ~ description    = "ocicloud" -> "creating first compartment"
    id             = "ocid1.compartment.oc1..aaaaaaaauqax7epejsqahhj6ax6uj6d5u7tbeyurmor75ikeo2yzc7qjsvpa"
  ~ name          = "Richajain" -> "test1"
  ~ timeouts      = {
    ~ delete = null
  }
}
```

- terraform apply**

```
Enter a value: yes

oci_identity_compartment.test: Modifying... [id=ocid1.compartment.oc1..aaaaaaaauqax7epejsqahhj6ax6uj6d5u7tbeyurmor75ikeo2yzc7qjsvpa]
oci_identity_compartment.test: Modifications complete after 6s [id=ocid1.compartment.oc1..aaaaaaaauqax7epejsqahhj6ax6uj6d5u7tbeyurmor75ikeo2yzc7qjsvpa]

Apply complete! Resources: 0 added, 1 changed, 0 destroyed.

Outputs:
compartment = {
  "compartment_id" = "ocid1.compartment.oc1..aaaaaaaep1rbcpqxpodimbh5befcepyfhd62fb44iifehqelgk3jnjdrla"
}
```

I have imported the previously made compartment in comp1 (you can notice the date here) . but now it is running in my new compartment comp1 which was created on 2nd nov.

Child Compartments

Create Compartment

Name	Status	OCID	Authorized	Security Zone ⓘ	Created
test1	● Active	...qjsvpa	Yes	-	Wed, Aug 30, 2023, 20:56:32 UTC ⋮

Showing 1 item < 1 of 1 >

PRACTICAL LESSON 5.2: CREATING A VIRTUAL CLOUD NETWORK

OBJECTIVE

In this practical, you use Terraform to create a virtual cloud network in your Oracle Cloud Infrastructure tenancy.

Prerequisites

- A **paid** Oracle Cloud Infrastructure account. See [Signing Up for Oracle Cloud Infrastructure](#).
- A MacOS, Linux, or Windows computer.

Tasks

1. In *<your-home-directory>*, create a directory called `tf-vcn` and change to that directory.

```
mkdir tf-vcn
```

```
cd tf-vcn
```

2. Create a file called `provider.tf`.

3. Add the following code to `provider.tf`:

- Replace the *fields with brackets*, with the correct information
- Add quotations around string values.

```
provider "oci" {  
  tenancy_ocid      = "ocid1.tenancy.oc1..aaaaaaaalwteck73qtntciw65iyqpd4uhg4edkz5n42t3jj  
jk72ps566fja"  
  user_ocid         = "ocid1.user.oc1..aaaaaaaumv2gh47s2f5goj3mpnh6qbifny2flb3powmpfb5q34  
nrqzcef6a"  
  private_key_path  = "/.apikey_new/oci_api_key_public.pem"  
  fingerprint       = "f2:f3:ef:02:fe:67:f7:f7:cc:be:32:a7:6e:94:9e:2e"  
  region            = "us-ashburn-1"  
}
```

4. Declare a Basic Network

Declare a basic network with an Oracle Cloud Infrastructure **virtual cloud network (VCN) module**, documented in the Terraform Registry. Then, run your scripts and create the network. In the next sections, add components to customize your network.

- I. Go to terraform Registry.

- II. Click **Browse Modules**.
- III. In the left navigation **Provider** list, click oci.
 - a. You get a list of Oracle Cloud Infrastructure provided modules.
Click **oracle-terraform-modules/vcn**.
- IV. For **Version**, from the list, click Version 3.1.0.
 - a. This tutorial uses version 3.1.0. If you use another version, you might have different number of required inputs and create different resources for your vcn.
- V. Create a file called vcn-module.tf.
- VI. Copy the code from **Provision Instructions** section of the page into vcn-module.tf.

5. Click **Inputs** and find all **Required Inputs**.

- Review the optional inputs to override.
- Add the following code to vcn-module.tf.

Snippet of the final vcn-module.tf code

```
# Source from https://registry.terraform.io/modules/oracle-terraform-
modules/vcn/oci/
module "vcn"{
  source = "oracle-terraform-modules/vcn/oci"
  version = "3.1.0"
  # insert the 5 required variables here

  # Required Inputs
  compartment_id = "ocid1.tenancy.oc1..aaaaaaaalwteck73qtntciw65iyqpd4uhg4edk
z5n42t3jjjk72ps566fja"
  region = "us-ashburn-1"

  internet_gateway_route_rules = null
  local_peering_gateways = null
  nat_gateway_route_rules = null

  # Optional Inputs
  vcn_name = "vcn-module"
  vcn_dns_label = "vcnmodule"
  vcn_cidrs = ["10.0.0.0/16"]

  create_internet_gateway = true
  create_nat_gateway = true
  create_service_gateway = true
}
```

6. In the tf-vcn directory, create a file called outputs.tf.
7. Add the following code to outputs.tf.

```
# Outputs for the vcn module

output "vcn_id" {
  description = "OCID of the VCN that is created"
  value = module.vcn.vcn_id
}
output "id-for-route-table-that-includes-the-internet-gateway" {
  description = "OCID of the internet-
route table. This route table has an internet gateway to be used for public su
bnets"
  value = module.vcn.ig_route_id
}
output "nat-gateway-id" {
  description = "OCID for NAT gateway"
  value = module.vcn.nat_gateway_id
}
output "id-for-for-route-table-that-includes-the-nat-gateway" {
  description = "OCID of the nat-route table -
This route table has a nat gateway to be used for private subnets. This route
table also has a service gateway."
  value = module.vcn.nat_route_id
}
```

Create your basic network with Terraform:

terraform init

terraform plan

terraform apply

When prompted for confirmation, enter yes, for your resources to be created.

After the virtual network is created, the outputs that you defined are displayed in the output terminal.

8. Watch the creation from the Console:

- Open the navigation menu and click **Networking**, and then click **Virtual Cloud Networks**.
- Select your compartment.
- Watch your virtual cloud network appear in the list of networks.

Congratulations! You have successfully created a basic virtual network using Terraform, in your Oracle Cloud Infrastructure account. You have a virtual network and you can be done at this point.

6. Practical Lesson 6 : Interact With Modules

PRACTICAL LESSON 6.1: SET UP A SIMPLE INFRASTRUCTURE WITH OCI TERRAFORM

OBJECTIVE

In this practical, you use Terraform to set up a simple infrastructure in your Oracle Cloud Infrastructure account.

Prerequisites

- A **paid** Oracle Cloud Infrastructure account. See [Signing Up for Oracle Cloud Infrastructure](#).
- A MacOS, Linux, or Windows computer.

Tasks

A. SET UP OCI TERRAFORM

1. In *<your-home-directory>*, create a directory called `tf-provider` and change to that directory.

```
mkdir tf-provider
```

```
cd tf-provider
```

2. Create a file called `provider.tf`.

3. Add the following code to `provider.tf`:

- Replace the *fields with brackets*, with the correct information
- Add quotations around string values.

```
provider "oci" {  
  tenancy_ocid      = "ocid1.tenancy.oc1..aaaaaaaalwteck73qntnciw65iyqpd4uhg4edkz5n42t3jj  
jk72ps566fja"  
  user_ocid         = "ocid1.user.oc1..aaaaaaaumv2gh47s2f5goj3mpnh6qbifny2flb3powmpfb5q34  
nrqzcef6a"  
  private_key_path  = "/.apikey_new/oci_api_key_public.pem"  
  fingerprint       = "f2:f3:ef:02:fe:67:f7:f7:cc:be:32:a7:6e:94:9e:2e"  
  region            = "us-ashburn-1"  
}
```

4. Save the `provider.tf` file.

5. In the `simple-infrastructure` directory, create a file called `availability-domains.tf`.

```
data "oci_identity_availability_domains" "ads" {
  compartment_id = "ocid1.tenancy.oc1..aaaaaaaelwteck73qtntciw65iyqpd4uhg4edkz5n42t3jjjk72ps566fja"
}
```

6. Add the following code to `outputs.tf`.

```
# Output the "list" of all availability domains.
output "all-availability-domains-in-your-tenancy" {
  value = data.oci_identity_availability_domains.ads.availability_domains
}
```

B. CREATING A COMPARTMENT

1. In the same directory. Create a file called `compartment.tf`

```
resource "oci_identity_compartment" "tf-compartment" {
  name          = "ExampleCompartment"
  description   = "This is an example compartment created with Terraform"
  compartment_id = "ocid1.tenancy.oc1..aaaaaaaelwteck73qtntciw65iyqpd4uhg4edkz5n42t3jjjk72ps566fja" # You can specify the parent compartment OCID here
}
```

2. Save the `compartment.tf` file.

3. Open the `output.tf` file and paste the code inside it.

```
# Outputs for compartment

output "compartment-name" {
  value = oci_identity_compartment.tf-compartment.name
}

output "compartment-OCID" {
  value = oci_identity_compartment.tf-compartment.id
}
```

C. FOR CREATING A COMPUTE INSTANCE

1. Create a file called `compute.tf`.

2. Add the following code to `compute.tf`.

```

resource "oci_core_instance" "linux"{
  # Required
  availability_domain = data.oci_identity_availability_domains.ads.availability_domains[0
].name
  compartment_id = "ocid1.tenancy.oc1..aaaaaaaalwteck73qntciw65iyqpd4uhg4edkz5n42t3jjjk
72ps566fja"
  shape = "VM.Standard.A1.Flex"
  source_details {
    source_id = "ocid1.image.oc1.iad.aaaaaaaamwu6pamu2qv33dcmsta5jp77fkmqjg5nb6zhigk3ch
lmeoqvt3wa"
    source_type = "image"
  }

  # Optional
  display_name = "demo"
  create_vnic_details {
    assign_public_ip = true
    subnet_id = "ocid1.subnet.oc1.iad.aaaaaaaab732bjgzo7opvedlqyea7ulhffokcshxsiml7arv6
ywuhkawchiq"
  }
  metadata = {
    ssh_authorized_keys = file("/home/ritesh/ssh-keys/ssh-keys.pub")
  }
  preserve_boot_volume = false

  shape_config {
    ocpus      = 2      # Number of OCPUs
    memory_in_gbs = 16 # Memory in GBs
  }
}

```

3. Open the output.tf file and paste the code inside it.

```

# Outputs for compute instance

output "public-ip-for-compute-instance" {
  value = oci_core_instance.linux.public_ip
}

```

D.FOR CREATING A VCN

4. Create a file called vcn-module.tf

5. Add the following code to vcn-module.tf

```

# Source from https://registry.terraform.io/modules/oracle-terraform-modules/vcn/oci/
module "vcn"{

```

```

source = "oracle-terraform-modules/vcn/oci"
version = "3.1.0"
# insert the 5 required variables here

# Required Inputs
compartment_id = "ocid1.tenancy.oc1..aaaaaaaelwteck73qtntciw65iyqpd4uhg4edkz5n42t3jjjk72ps566fja"
region = "us-ashburn-1"

internet_gateway_route_rules = null
local_peering_gateways = null
nat_gateway_route_rules = null

# Optional Inputs
vcn_name = "vcn-module"
vcn_dns_label = "vcnmodule"
vcn_cidrs = ["10.0.0.0/16"]

create_internet_gateway = true
create_nat_gateway = true
create_service_gateway = true
}

```

6. Open the output.tf file and paste the code inside it.

```

# Outputs for the vcn module

output "vcn_id" {
  description = "OCID of the VCN that is created"
  value = module.vcn.vcn_id
}

output "id-for-route-table-that-includes-the-internet-gateway" {
  description = "OCID of the internet-
route table. This route table has an internet gateway to be used for public subnets"
  value = module.vcn.ig_route_id
}

output "nat-gateway-id" {
  description = "OCID for NAT gateway"
  value = module.vcn.nat_gateway_id
}

output "id-for-for-route-table-that-includes-the-nat-gateway" {
  description = "OCID of the nat-route table -
This route table has a nat gateway to be used for private subnets. This route table also h
as a service gateway."
  value = module.vcn.nat_route_id
}

```

D. RUN THE SCRIPTS

- Initialize a working directory in the tf-simple-infrastructure directory.

terraform init

- Create an **execution plan** and review the changes that Terraform plans to make to your account:

terraform plan

- Create your simple infrastructure with Terraform:

terraform plan

When prompted for confirmation, enter yes, for your resources to be created.

E. WATCH THE CREATION IN THE CONSOLE

1. Open the navigation menu and click **Identity & Security**. Under **Identity**, click **Compartments**.
2. Refresh the page, until you see the compartment name.
3. Open the navigation menu click **Networking**, and then click **Virtual Cloud Networks**.
4. In the left navigation of the **Virtual Cloud Networks** page, find your compartment.
5. If you can't find your compartment, then refresh the left navigation.
6. Go to your compartment and watch your virtual cloud network appear in the list of networks.
7. Click <your-vcn-name>.
8. Review the created **Resources**.
9. Open the navigation menu and click **Compute**. Under **Compute**, click **Instances**.
10. Watch your instance appear in the list of instances.

7. Practical Lesson 7 : Use the core Terraform workflow

PRACTICAL LESSON 7.1 : CREATE A KUBERNETES CLUSTER

OBJECTIVE

In this tutorial, you use Terraform to set up a Kubernetes cluster in your Oracle Cloud Infrastructure account.

Prerequisites

- A **paid** Oracle Cloud Infrastructure account. See [Signing Up for Oracle Cloud Infrastructure](#).
- A MacOS, Linux, or Windows computer.
- **Keep** the scripts you created in the following directory: \$HOME/tf-simple-infrastructure/

Tasks

1. In your \$HOME directory, create a directory called tf-cluster and change to that directory.

```
mkdir tf-cluster
```

```
cd tf-cluster
```

2. Copy all the files ending in .tf from the tf-simple-infrastructure directory.

```
cp ../tf-simple-infrastructure/*.tf .
```

3. Remove the vcn-module.tf file and remove the contents of VCN-output which you have done it Section D point no. 3

4. Create a file called `vcn-module.tf`.

- Go to [Terraform Registry](#).
- Click **Browse Modules**.
- In the left navigation **Provider** list, click `oci`. You get a list of Oracle Cloud Infrastructure provided modules.
- Click **oracle-terraform-modules/vcn**.
- For **Version**, from the list, click `Version 3.1.0`.

This practical uses version 3.1.0. If you use another version, you might have a different number of required inputs and create different resources for your vcn.

5. Copy the code from **Provision Instructions** section of the page into `vcn-module.tf`.
6. Click **Inputs** and find all **Required Inputs**.
7. Review the optional inputs to override.
8. Add the following code to `vcn-module.tf`.
 - Replace `<compartment-ocid>` and `<region-identifier>` with the information in the **Gather Required Information** section.
 - Replace `<your-vcn-name>` with a name of your choice. The default value is `vcn-module`
 - Replace `<your-dns-label>` with a label of your choice. The default value is `vcnmodule`.
9. Save the `vcn-module.tf` file.
10. Add **output** blocks to your code to get information about your virtual cloud network after you run your scripts.
11. Add the following code to `outputs.tf`.

```
# Outputs for the vcn module

output "vcn_id" {
  description = "OCID of the VCN that is created"
  value = module.vcn.vcn_id
}

output "id-for-route-table-that-includes-the-internet-gateway" {
  description = "OCID of the internet-
route table. This route table has an internet gateway to be used for public subnets"
  value = module.vcn.ig_route_id
}

output "nat-gateway-id" {
  description = "OCID for NAT gateway"
  value = module.vcn.nat_gateway_id
}

output "id-for-for-route-table-that-includes-the-nat-gateway" {
  description = "OCID of the nat-route table -
This route table has a nat gateway to be used for private subnets. This route table also h
as a service gateway."
  value = module.vcn.nat_route_id
}
```

Create scripts for security lists, private, and public subnets to create the same virtual network as the Console's wizard.

1. Now, we will create security list for public and private subnet

2. Create a file called `private-security-list.tf`.

3. Add the following code to `private-security-list.tf`:

```
resource "oci_core_security_list" "private-security-list" {
  #Required
  compartment_id = "ocid1.tenancy.oc1..aaaaaaaalwteck73qtntciw65iyqpd4uhg4edkz5n42t3jjjk72ps566fja"
  vcn_id = module.vcn.vcn_id

  egress_security_rules {
    stateless = false
    destination = "0.0.0.0/0"
    destination_type = "CIDR_BLOCK"
    protocol = "all"
  }

  ingress_security_rules {
    stateless = false
    source = "10.0.0.0/16"
    source_type = "CIDR_BLOCK"
    # Get protocol numbers from https://www.iana.org/assignments/protocol-numbers/protocol-numbers.xhtml TCP is 6
    protocol = "6"
    tcp_options {
      min = 22
      max = 22
    }
  }

  ingress_security_rules {
    stateless = false
    source = "0.0.0.0/0"
    source_type = "CIDR_BLOCK"
    # Get protocol numbers from https://www.iana.org/assignments/protocol-numbers/protocol-numbers.xhtml ICMP is 1
    protocol = "1"

    # For ICMP type and code see: https://www.iana.org/assignments/icmp-parameters/icmp-parameters.xhtml
    icmp_options {
      type = 3
      code = 4
    }
  }
}
```

```

ingress_security_rules {
  stateless = false
  source = "10.0.0.0/16"
  source_type = "CIDR_BLOCK"
  # Get protocol numbers from https://www.iana.org/assignments/protocol-
  numbers/protocol-numbers.xhtml ICMP is 1
  protocol = "1"

  # For ICMP type and code see: https://www.iana.org/assignments/icmp-parameters/icmp-
  parameters.xhtml
  icmp_options {
    type = 3
  }
}
}

```

4. Add the following code to `outputs.tf`.

```

# Outputs for private security list

output "private-security-list-name" {
  value = oci_core_security_list.private-security-list.display_name
}

output "private-security-list-OCID" {
  value = oci_core_security_list.private-security-list.id
}

```

Create a Security List for the Public Subnet

In this section, you create a security list in your network with egress and ingress rules. **Later**, you assign this security list to a **public subnet**.

1. In the home directory, copy the `private-security-list.tf` file and call it `public-security-list.tf`.

```
cp private-security-list.tf public-security-list.tf
```

2. Open the `public-security-list.tf` file and **update** the following:

- **resource block local name:** from `"private-security-list"` to **`"public-security-list"`**
- **security list name:** `display_name = "security-list-for-public-subnet"`

```

resource "oci_core_security_list" "public-security-list" {
  #Required
  compartment_id = "ocid1.tenancy.oc1..aaaaaaaawteck73qtntciw65iyqpd4uh
g4edkz5n42t3jjjk72ps566fja"
  vcn_id = module.vcn.vcn_id

  egress_security_rules {
    stateless = false
    destination = "0.0.0.0/0"
    destination_type = "CIDR_BLOCK"
    protocol = "all"
  }

  ingress_security_rules {
    stateless = false
    source = "0.0.0.0/0"
    source_type = "CIDR_BLOCK"
    # Get protocol numbers from https://www.iana.org/assignments/protocol
-numbers/protocol-numbers.xhtml TCP is 6
    protocol = "6"
    tcp_options {
      min = 22
      max = 22
    }
  }
}

ingress_security_rules {
  stateless = false
  source = "0.0.0.0/0"
  source_type = "CIDR_BLOCK"
  # Get protocol numbers from https://www.iana.org/assignments/protocol
-numbers/protocol-numbers.xhtml ICMP is 1
  protocol = "1"

  # For ICMP type and code see: https://www.iana.org/assignments/icmp-
parameters/icmp-parameters.xhtml
  icmp_options {
    type = 3
    code = 4
  }
}

ingress_security_rules {

```

```

stateless = false
source = "10.0.0.0/16"
source_type = "CIDR_BLOCK"
# Get protocol numbers from https://www.iana.org/assignments/protocol-
numbers/protocol-numbers.xhtml ICMP is 1
protocol = "1"

# For ICMP type and code see: https://www.iana.org/assignments/icmp-
parameters/icmp-parameters.xhtml
icmp_options {
  type = 3
}
}
}

```

3. Add the following code to `outputs.tf`.

```

# Outputs for public security list

output "public-security-list-name" {
  value = oci_core_security_list.public-security-list.display_name
}
output "public-security-list-OCID" {
  value = oci_core_security_list.public-security-list.id
}

```

CREATE A PRIVATE SUBNET

In this section, you create a **private subnet** in your network and associate the **private security list** to this subnet. You also add the **NAT route table** that you made with the `vcn` module to this subnet. The NAT route table has one NAT gateway and one service gateway and is designed for private subnets. See the first diagram in the tutorial.

1. In the home directory, create a file called `private-subnet.tf` and add the following code to it

```

# Source from https://registry.terraform.io/providers/oracle/oci/latest/doc
s/resources/core_subnet

resource "oci_core_subnet" "vcn-private-subnet" {

  # Required
  compartment_id = "ocid1.tenancy.oc1..aaaaaaaalwteck73qtntciw65iyqpd4uhg4
edkz5n42t3jjjk72ps566fja"
}

```

```

vcn_id = module.vcn.vcn_id
cidr_block = "10.0.1.0/24"

# Optional
# Caution: For the route table id, use module.vcn.nat_route_id.
# Do not use module.vcn.nat_gateway_id, because it is the OCID for the gateway and not the route table.
route_table_id = module.vcn.nat_route_id
security_list_ids = [oci_core_security_list.private-security-list.id]
display_name = "private-subnet"
}

```

3. Add the following code to `outputs.tf`.

```

# Outputs for private subnet

output "private-subnet-name" {
  value = oci_core_subnet.vcn-private-subnet.display_name
}
output "private-subnet-OCID" {
  value = oci_core_subnet.vcn-private-subnet.id
}

```

CREATING A PUBLIC SUBNET

In this section, you create a **public subnet** in your network and associate the **public security list** to this subnet. You also add the **internet route table** that you made with the VCN module to this subnet. The internet route table has an internet gateway and is designed for public subnets. See the first diagram in the tutorial.

1. In the home directory, create a file called `public-subnet.tf` and add the following code to it:

```

# Source from https://registry.terraform.io/providers/oracle/oci/latest/docs/resources/core_subnet

resource "oci_core_subnet" "vcn-public-subnet"{

  # Required
  compartment_id = "ocid1.tenancy.oc1..aaaaaaaalwteck73qntciw65iyqpd4uhg4edkz5n42t3jjjk72ps566fja"
  vcn_id = module.vcn.vcn_id
  cidr_block = "10.0.0.0/24"

  # Optional
  route_table_id = module.vcn.ig_route_id
  security_list_ids = [oci_core_security_list.public-security-list.id]
}

```

```
display_name = "public-subnet"
}
```

3. Add the following code to `outputs.tf`.

```
# Outputs for public subnet

output "public-subnet-name" {
  value = oci_core_subnet.vcn-public-subnet.display_name
}
output "public-subnet-OCID" {
  value = oci_core_subnet.vcn-public-subnet.id
}
```

Declare a Cluster

1. Create a file called `cluster.tf`.
2. Add the following code to `cluster.tf`.

Replace `<your-cluster-name>` with a name of your choice. Example: `tf-cluster`.

```
# Source from https://registry.terraform.io/providers/oracle/oci/latest/docs/resources/containerengine_cluster

resource "oci_containerengine_cluster" "oke-cluster" {
  # Required
  compartment_id = oci_identity_compartment.tf-compartment.id
  kubernetes_version = "v1.25.4"
  name = "tf-cluster"
  vcn_id = module.vcn.vcn_id
}
```

- Create a file called `node-pool.tf`.
- Add the following code to `node-pool.tf`.
 - Replace the following fields with the information you gathered in section one:
 - `<node-shape>` with **VM.Standard2.1**
 - `<image-ocid>`
 - Replace the following field with the name you chose when you declared a cluster:
 - `<your-cluster-name>`

```
resource "oci_containerengine_node_pool" "oke-node-pool" {
  # Required
  cluster_id = oci_containerengine_cluster.oke-cluster.id
  compartment_id = oci_identity_compartment.tf-compartment.id
}
```

```

kubernetes_version = "v1.25.4"
name = "pool1"
node_config_details{
  placement_configs{
    availability_domain = data.oci_identity_availability_domains.ad
s.availability_domains[0].name
    subnet_id = oci_core_subnet.vcn-private-subnet.id
  }
  placement_configs{
    availability_domain = data.oci_identity_availability_domains.ad
s.availability_domains[1].name
    subnet_id = oci_core_subnet.vcn-private-subnet.id
  }
  placement_configs{
    availability_domain = data.oci_identity_availability_domains.ad
s.availability_domains[2].name
    subnet_id = oci_core_subnet.vcn-private-subnet.id
  }
  size = 3
}
node_shape = "VM.Standard2.1"

# Using image Oracle-Linux-7.x-<date>
# Find image OCID for your region from https://docs.oracle.com/iaas/ima
ges/
node_source_details {
  image_id = "ocid1.image.oc1.iad.aaaaaaaavbafqm6xn5bkhrrqtohcphv5b7
c7wzile7w33cv4drdiittiivpa"
  source_type = "image"
}

# Optional
initial_node_labels {
  key = "name"
  value = "tf-cluster"
}
}

```

3. Add the following code to `outputs.tf`.

```

# Outputs for k8s cluster

output "cluster-name" {
  value = oci_containerengine_cluster.oke-cluster.name
}
output "cluster-OCID" {
  value = oci_containerengine_cluster.oke-cluster.id
}

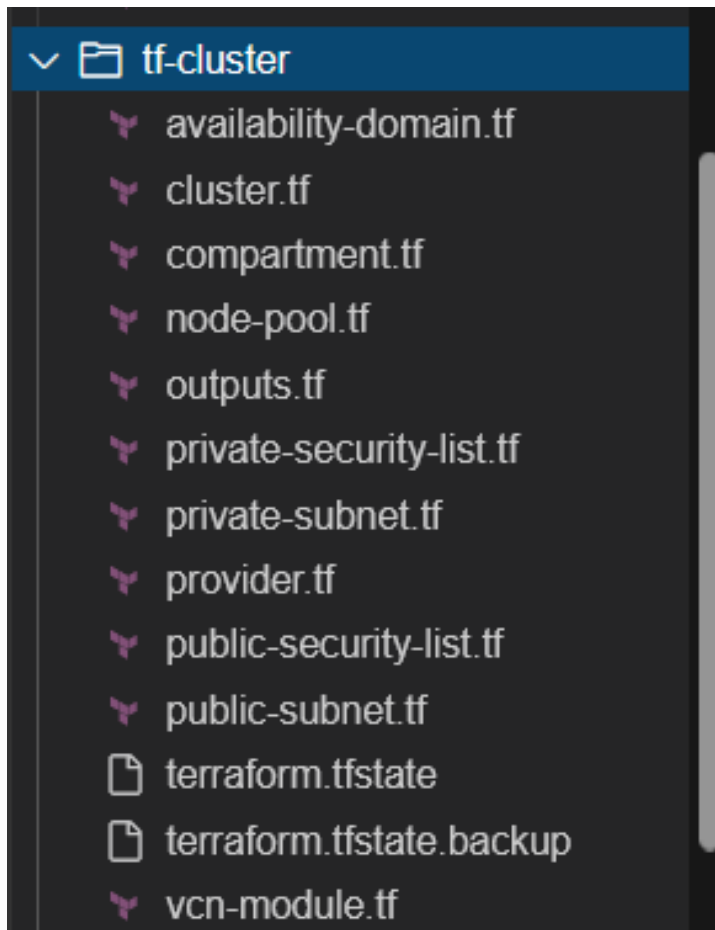
```

```
output "cluster-kubernetes-version" {
  value = oci_containerengine_cluster.oke-cluster.kubernetes_version
}
output "cluster-state" {
  value = oci_containerengine_cluster.oke-cluster.state
}

# Outputs for k8s node pool

output "node-pool-name" {
  value = oci_containerengine_node_pool.oke-node-pool.name
}
output "node-pool-OCID" {
  value = oci_containerengine_node_pool.oke-node-pool.id
}
output "node-pool-kubernetes-version" {
  value = oci_containerengine_node_pool.oke-node-pool.kubernetes_version
}
output "node-size" {
  value = oci_containerengine_node_pool.oke-node-pool.node_config_details[0].size
}
output "node-shape" {
  value = oci_containerengine_node_pool.oke-node-pool.node_shape
}
```

Note: Make sure that you have all the .tf files as given in the snapshot



RUN THE SCRIPTS

- Initialize a working directory in the tf-simple-infrastructure directory.

```
terraform init
```

- Create an **execution plan** and review the changes that Terraform plans to make to your account:

```
terraform plan
```

- Create your simple infrastructure with Terraform:

```
terraform plan
```

When prompted for confirmation, enter yes, for your resources to be created.

Congratulations! You have created a Kubernetes cluster using Terraform, in your Oracle Cloud Infrastructure account.

8. Practical Lesson 8: Implement And Maintain State

PRACTICAL LESSON 8.1: Oracle Cloud Infrastructure (OCI) Autonomous Database Warehouse (ADW)

Objective: in this, we will create an autonomous Database warehouse using terraform

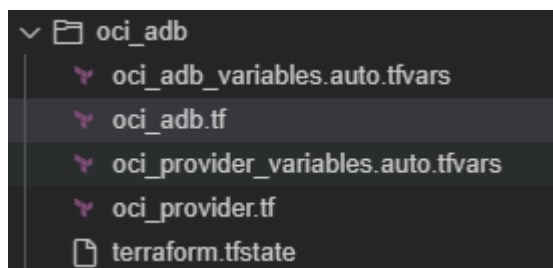
Prerequisites:

- A paid/free Oracle Cloud Infrastructure account. See [Signing Up for Oracle Cloud Infrastructure](#).
- A MacOS, Linux, or Windows computer.

Tasks

- Create Working Directory
- oci_adb.tf
- oci_adb_variables.auto.tfvars
- Build the OCI Autonomous Database

Note : make sure to include all these below .tf files in your working directory



1. Create a working directory , Mkdir oci_adb
2. Move to that directory, Cd oci_adb
3. In this directory, Create the same oci_provider.tf and oci_provider.auto.tfvars file (created earlier)
4. Create a file called "oci_adb.tf" with the following code

```
• # Variables
• variable "compartment_id" { type = string }
• variable "db_name"        { type = string }
• variable "admin_password" { type = string }
• variable "db_version"     { type = string }
• variable "db_workload"    { type = string }
•
• variable "is_free_tier"    { type = string }
```

```

•
•   variable "license_model" { type = string }
•
•   variable "cpu_core_count" {
•     type      = number
•     default = 1
•   }
•
•   variable "data_storage_size_in_tbs" {
•     type      = number
•     default = 1
•   }
•   # Resources
•   resource "oci_database_autonomous_database" "tf_adb" {
•     compartment_id      = var.compartment_id
•     cpu_core_count      = var.cpu_core_count
•     data_storage_size_in_tbs = var.data_storage_size_in_tbs
•     db_name              = var.db_name
•     admin_password       = var.admin_password
•     db_version           = var.db_version
•     db_workload          = var.db_workload
•     display_name         = var.db_name
•     is_free_tier         = var.is_free_tier
•     license_model        = var.license_model
•   }
•
•   # Outputs
•   output "db_name" {
•     value = oci_database_autonomous_database.tf_adb.display_name
•   }
•
•   output "db_state" {
•     value = oci_database_autonomous_database.tf_adb.state
•   }

```

6. create a file named **oci_adb.auto.tfvars** with below code

```

compartment_id = "ocid1.compartment.oc1..aaaaaaaaplrbcpngxopdimbh5befcepyfhd62
fb44iifehqelgk3jnjodrla"
db_name        = "obadw2"
admin_password = "MyStrongPassword123"
db_version     = "19c"

db_workload    = "DW"

is_free_tier   = "true"
license_model  = "LICENSE_INCLUDED"

```

9. Initialise your working directory /scripts with terraform init
10. terraform plan
11. terraform apply

Autonomous Databases

Autonomous Databases *in comp1 Compartment*

Autonomous Database delivers fast performance and requires no database administration. It performs all routine database maintenance tasks without human intervention while the system is running. [Learn more](#).

Create Autonomous Database

Display name	State	Compute	Storage	Workload type	Disaster recovery	Created	
obadw2 Always Free	Available	Included ⓘ	Included ⓘ	Data Warehouse	—	Sat, Nov 4, 2023, 06:07:25 UTC	⋮

Displaying 1 Autonomous Database < 1 of 1 >

9. Practical Lesson 9: Read, generate and modify configuration

Practical Lesson 9.1: CREATING AN OBJECT STORAGE BUCKET ON OCI

this Activity describes you how to create an object storage bucket on Oracle Cloud Infrastructure (OCI) using terraform.

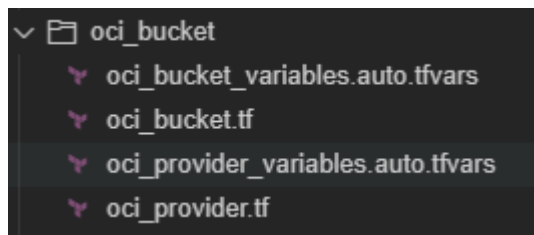
Pre-requisites:

- An oracle cloud account
- Mac os, Linux or windows operating System

Tasks:

- Create Working Directory
- oci_bucket.tf
- oci_bucket_variables.auto.tfvars
- Build the OCI Object Storage Bucket

Note: make sure to include below .tf files



1. Create a working directory , Mkdir oci_bucket
2. Move to that directory, Cd oci_bucket
3. In this directory, Create the same oci_provider.tf and oci_provider.auto.tfvars file (created earlier)
4. Create a file called "oci_bucket.tf" with the following code

```
# Variables
variable "compartment_id" { type = string }
variable "bucket_name"    { type = string }
variable "bucket_namespace" { type = string }

variable "bucket_access_type" {
  type    = string
  default = "NoPublicAccess"
}
```

```
# Resources
resource "oci_objectstorage_bucket" "tf_bucket" {
  compartment_id = var.compartment_id
  name           = var.bucket_name
  namespace      = var.bucket_namespace
  access_type    = var.bucket_access_type
}

# Outputs
output "bucket_name" {
  value = oci_objectstorage_bucket.tf_bucket.name
}

output "bucket_id" {
  value = oci_objectstorage_bucket.tf_bucket.bucket_id
}
```

5. create a file named **"oci_bucket_variables.auto.tfvars"**

```
compartment_id = "ocid1.compartment.oc1..aaaaaaaaplrbcpqxdimh5befcepyfhd62fb44iifehqelgk3jnjodrla"
bucket_name    = "ob-bucket2"
bucket_namespace = "bm4yz1c6nn1k"
```

6. terraform init

7. terraform plan

8. terraform apply

Outputs:

```
bucket_id = "ocid1.bucket.oc1.ap-mumbai-1.aaaaaac7dhhfjz1lqdam4jwui7gjjvfdnbnzaznh5spyhkusxp2lk1bxbpq"
bucket_name = "ob-bucket2"
richajain1@cloudshell:oci_bucket (ap-mumbai-1)$
```

Buckets in comp1 *Compartment*

Object Storage provides unlimited, high-performance, durable, and secure data storage. Data is uploaded as objects that are stored in buckets. [Learn more](#)

i You are using approximately **0 bytes** of the **20 GiB** limit of free combined Object Storage and Archive Storage. [Upgrade](#) to use unlimited storage. [Show details](#)

Create Bucket

Name	Default Storage Tier	Visibility	Created
ob-bucket2	Standard	Private	Sat, Nov 4, 2023, 07:29:44 UTC

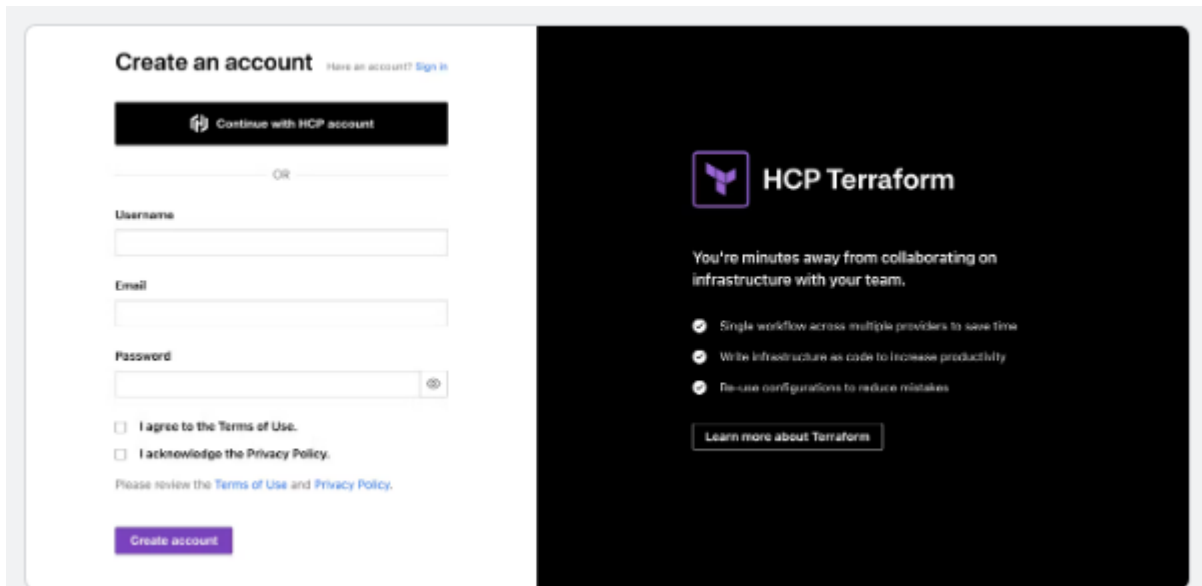
Showing 1 item < 1 of 1 >

10. Practical Lesson 10: HCP Terraform Capabilities

Practical Lesson 10.1: sign up Terraform Cloud

1. Create an account

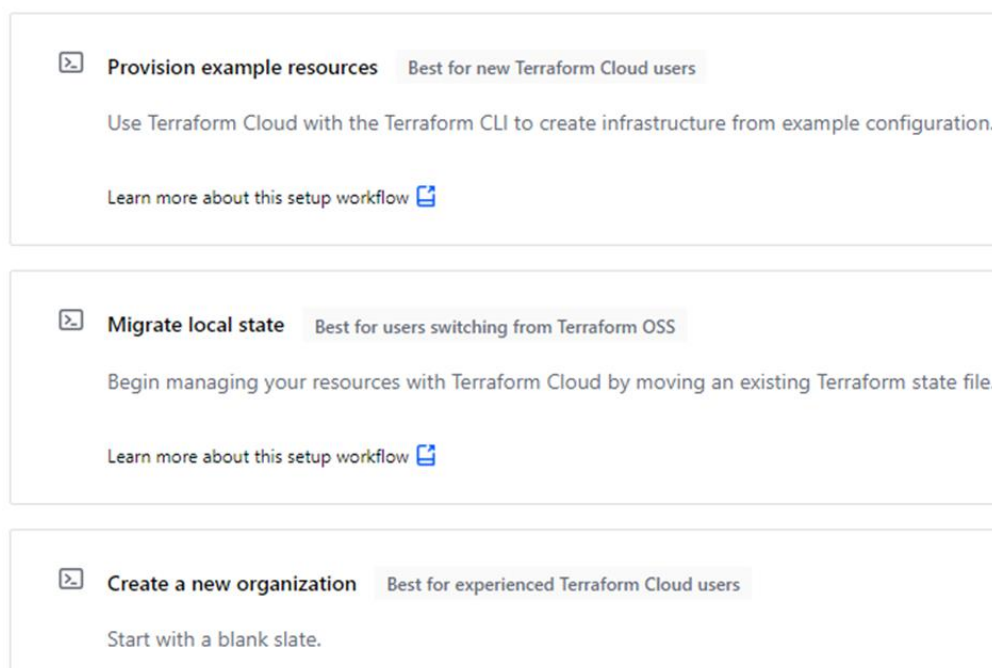
- Go to <https://app.terraform.io/signup/account> and follow the prompts to create a free HCP Terraform account.



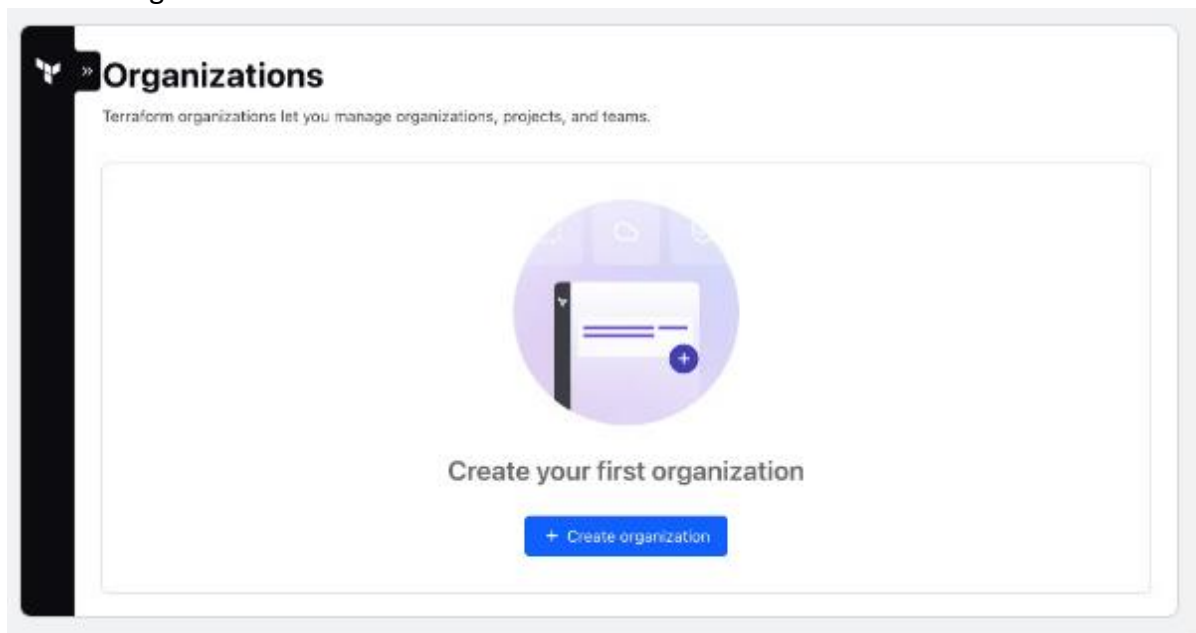
- When you sign up, you will receive an email asking you to confirm your email address. Confirm your email address before moving on. When you click the link to confirm your email address,

Welcome to Terraform Cloud!

Choose your setup workflow



Click Create organization.



- **Create an organization**

The next screen will prompt you to create a new organization. Creating organizations of up to 5 users is free, and the members you add to the organization will be able to collaborate on your workspaces and share private modules and providers.

Enter an organization name and email address. You can use the same email address that you used for your account.

A screenshot of the 'Create a new organization' form in HCP Terraform. The page has a dark sidebar with a logo and a breadcrumb '» Organizations / New'. The main heading is 'Create a new organization'. Below it is a subtitle 'Organizations are privately shared spaces for teams to collaborate on infrastructure.' and a link 'Learn more' about organizations in HCP Terraform. There are two input fields: 'Organization name' and 'Email address'. Below the 'Organization name' field is a note: 'Organization names must be unique and can only include numbers, letters, underscores (_), and hyphens (-)'. Below the 'Email address' field is a note: 'The organization email is used for any future notifications, such as billing alerts, and the organization avatar, via [gravatar.com](#)'. At the bottom is a blue button labeled 'Create organization'.

Click **Create organization**. In this tutorial, you created an HCP Terraform account and organization

Practical Lesson 10.2 : Log In To HCP Terraform From The CLI

HCP Terraform runs Terraform operations and stores state remotely, so you can use Terraform without worrying about the stability of your local machine, or the security of your state file.

To use HCP Terraform from the command line, you must log in. Logging in allows you to trigger remote plans and runs, migrate state to the cloud, and perform other remote operations on configurations with HCP Terraform.

Pre-requisites

- The Terraform CLI (version 1.1.0+) installed locally
- An HCP Terraform account

Start the login flow

In order to authenticate with HCP Terraform, run the terraform login subcommand. Respond yes to the prompt to confirm that you want to authenticate.

```
Administrator: Command Prompt - terraform login
Microsoft Windows [Version 10.0.19045.4170]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\system32>terraform version
Terraform v1.6.1
on windows_amd64

Your version of Terraform is out of date! The latest version
is 1.7.5. You can update by downloading from https://www.terraform.io/downloads.html

C:\Windows\system32>terraform login
Terraform will request an API token for app.terraform.io using your browser.

If login is successful, Terraform will store the token in plain text in
the following file for use by subsequent commands:
  C:\Users\Admin\AppData\Roaming\terraform.d\credentials.tfrc.json

Do you want to proceed?
  Only 'yes' will be accepted to confirm.

Enter a value:
```

```
the following file for use by subsequent commands:
  C:\Users\Admin\AppData\Roaming\terraform.d\credentials.tfrc.json

Do you want to proceed?
  Only 'yes' will be accepted to confirm.

Enter a value: yes

-----

Terraform must now open a web browser to the tokens page for app.terraform.io.

If a browser does not open this automatically, open the following URL to proceed:
  https://app.terraform.io/app/settings/tokens?source=terraform-login

-----

Generate a token using your browser, and copy-paste it into this prompt.

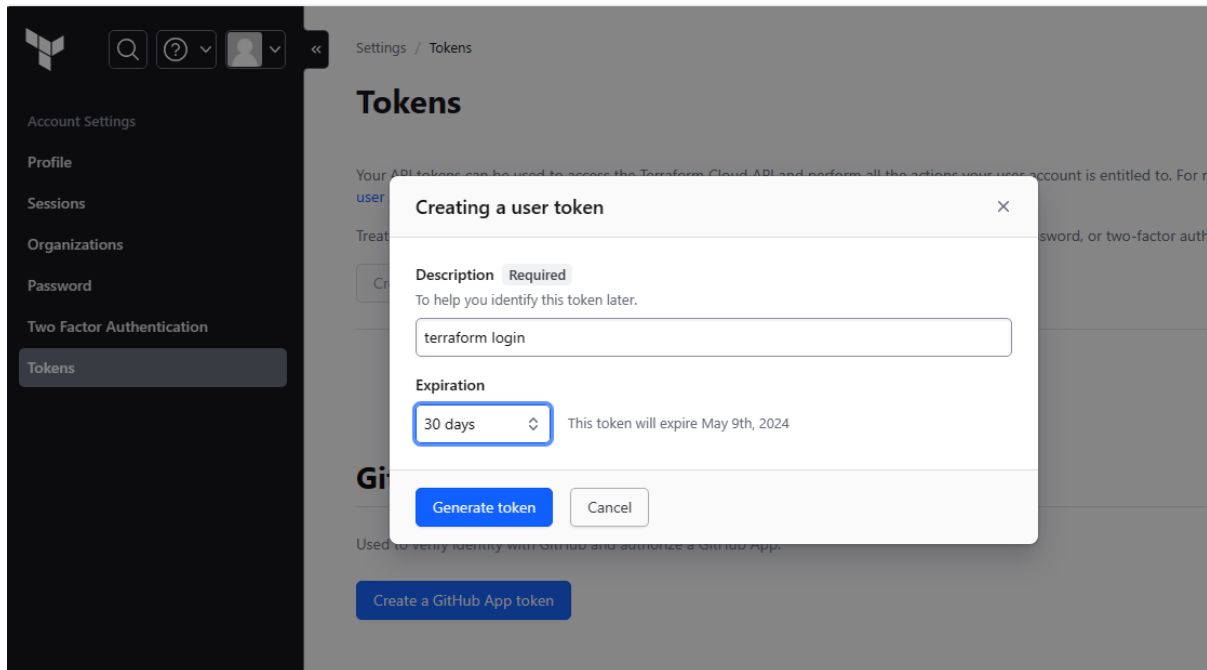
Terraform will store the token in plain text in the following file
for use by subsequent commands:
  C:\Users\Admin\AppData\Roaming\terraform.d\credentials.tfrc.json

Token for app.terraform.io:
Enter a value:
```

after entering yes, the browser window will automatically appears and will ask to generate token. if does not open automatically then , follow the URL on the screen to open .

- **Generate a token**

A browser window will automatically open to the HCP Terraform login screen. Enter a token name in the web UI, or leave the default name, terraform login.



Tokens (1)

terraform login 

 Expires May 9th 2024

 Copy the token, then return to your terminal and paste it into your Terraform login prompt. Terraform Cloud will not display this token again, so store it securely.

`xDh12pF59HfWZQ.atlasv1.LIUcIbElorGF5NTVavtHfjymvpZvg9DXFmah5GcX92k1VI3A5CQMMmEEMQSD7c1cXko` 

 Created **just now** by user  Richa Jain  Token has not been used

Save a copy of the token in a secure location. It provides access to your HCP Terraform organization. Terraform will also store your token locally at the file path specified in the command output. save and use this token now to authenticate the CLI.

3. Add the token to the CLI

When the Terraform CLI prompts you, paste the user token exactly once into your terminal. Terraform will hide the token for security when you paste it into your terminal. Press Enter to complete the authentication process.

```
if a browser does not open this automatically, open the following URL to proceed:  
https://app.terraform.io/app/settings/tokens?source=terraform-login
```

```
-----  
Generate a token using your browser, and copy-paste it into this prompt.
```

```
Terraform will store the token in plain text in the following file  
for use by subsequent commands:
```

```
  C:\Users\Admin\AppData\Roaming\terraform.d\credentials.tfrc.json
```

```
Token for app.terraform.io:
```

```
Enter a value:
```

```
Retrieved token for user Richa_jain
```

```
-----  
Welcome to Terraform Cloud!
```

```
Documentation: terraform.io/docs/cloud
```

```
New to TFC? Follow these steps to instantly apply an example configuration:
```

```
$ git clone https://github.com/hashicorp/tfc-getting-started.git
```

```
$ cd tfc-getting-started
```

Once you have authenticated to HCP Terraform, you are ready to perform remote operations.

Practical Lesson 10.3: Create a Credential Set

By defining a variable set for your credentials, you can easily reuse the same variables across multiple workspaces and efficiently and securely rotate your credentials. In this tutorial, you will create a variable set for your AWS credentials.

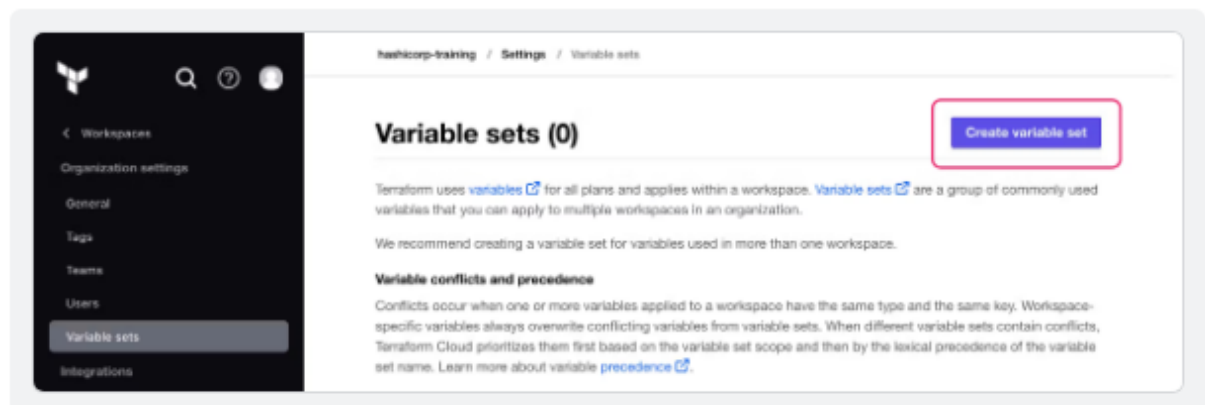
Prerequisites

This tutorial requires:

- an HCP Terraform account and organization
- an AWS account

Create a variable set

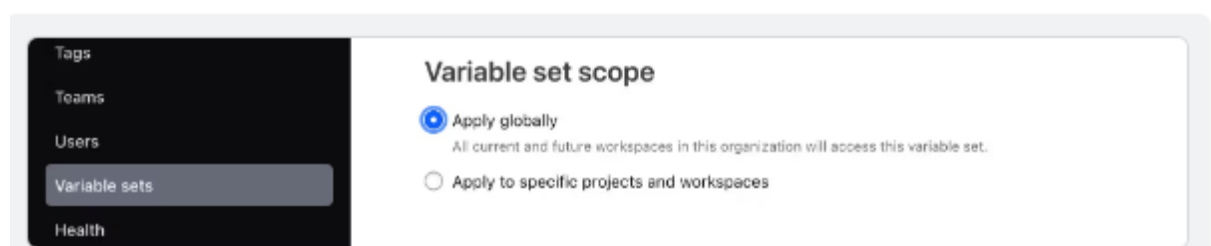
You can define both input variables and environment variables in variable sets. Input variables define the values for variables you reference in your configuration, while environment variables typically store provider credentials or modify Terraform's behavior, such as logging verbosity. To create a variable set, navigate to your organization's Settings, then to Variable sets. Next, click Create variable set.



Name the variable set "AWS Credentials".

You can apply a variable set to all workspaces in your organization, or scope it to specific workspaces or projects. When using variable sets with credentials, reuse the variable set with care and avoid the global scope, since it does not follow least privilege best practices.

select Apply globally as the variable set scope.



Next, define your AWS credentials as environment variables. Use an existing Access key ID and Secret access key or create a new pair in the IAM console. Click +Add Variable. Select the Environment variable option. Set the key to `AWS_ACCESS_KEY_ID` and the value to your AWS Access Key ID. Mark it as Sensitive and click Add variable.

Variables

You can add any number of [Terraform](#) and [Environment](#) variables. Terraform will use these variables for all plan and apply operations in the specified workspaces.

Key	Value	Category
Select variable category		
☐ Terraform variable These variables should match the declarations in your configuration. Click the HCL box to use interpolation or set a non-string value.		
☒ Environment variable These variables are available in the Terraform runtime environment.		
Key AWS_ACCESS_KEY_ID	Value [REDACTED]	☒ Sensitive
Variable Description description (optional)		
<button>Add variable</button> <button>Cancel</button>		

Now, click + Add Variable again. Define another environment variable named `AWS_SECRET_ACCESS_KEY` and set it to your AWS Secret access key. Mark it as Sensitive and click Add variable.

Now click Create variable set.

Variables

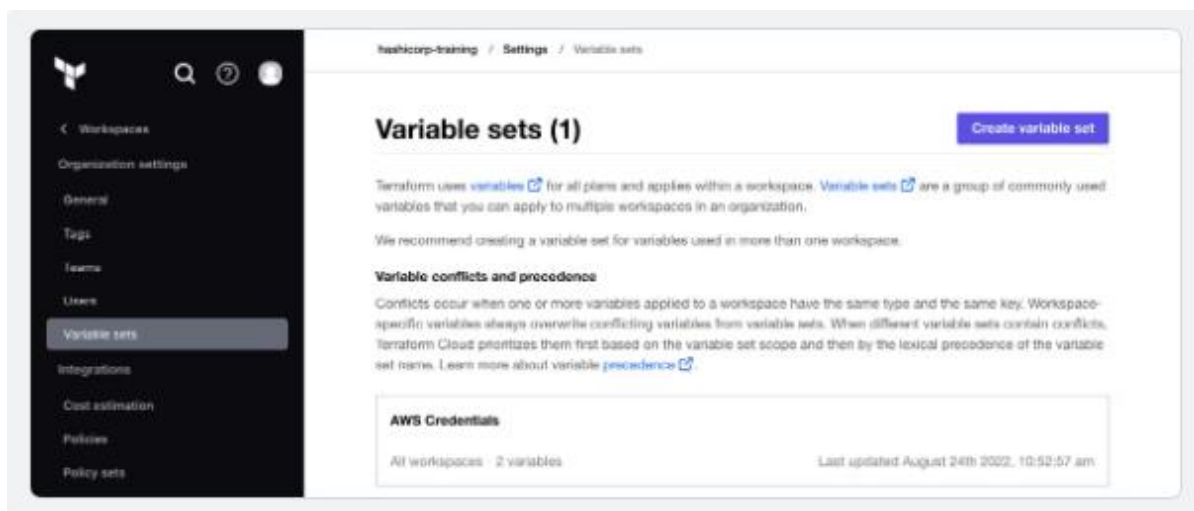
You can add any number of [Terraform](#) and [Environment](#) variables. Terraform will use these variables for all plan and apply operations in the specified workspaces.

Key	Value	Category
AWS_ACCESS_KEY_ID SENSITIVE	Sensitive - write only	env
AWS_SECRET_ACCESS_KEY SENSITIVE	Sensitive - write only	env

+ Add variable

Save variable set Delete variable set

HCP Terraform now lists your new variable set on the Variable Sets page. It also shows the number of variables it contains and indicates that the set applies to all workspaces, since you selected the global scope.



In this tutorial, you created a global variable set

Practical Lesson 10.4: Create a Workspace

Once you have created an HCP Terraform account and created or joined an organization, you can start managing infrastructure with HCP Terraform. In this tutorial, you will create a CLI-driven workspace.

Prerequisites

While Terraform can provision resources on many different providers and connect with several popular version control systems (VCSs), this tutorial requires:

- an HCP Terraform account and organization
- Terraform v1.1+ installed locally and configured with your HCP Terraform token
- an AWS account
- a GitHub account

Tasks :

1. Clone a Github repository

Clone the example GitHub repository for this tutorial, which contains the Terraform configuration that defines an AWS EC2 instance and its attributes.

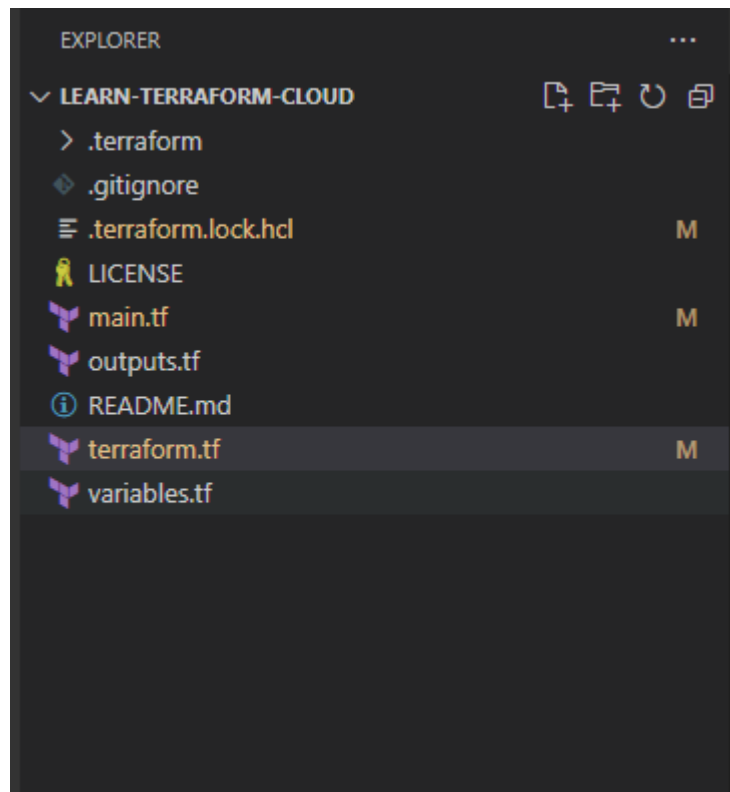
```
$ git clone https://github.com/hashicorp/learn-terraform.git
```

Change to the repository directory.

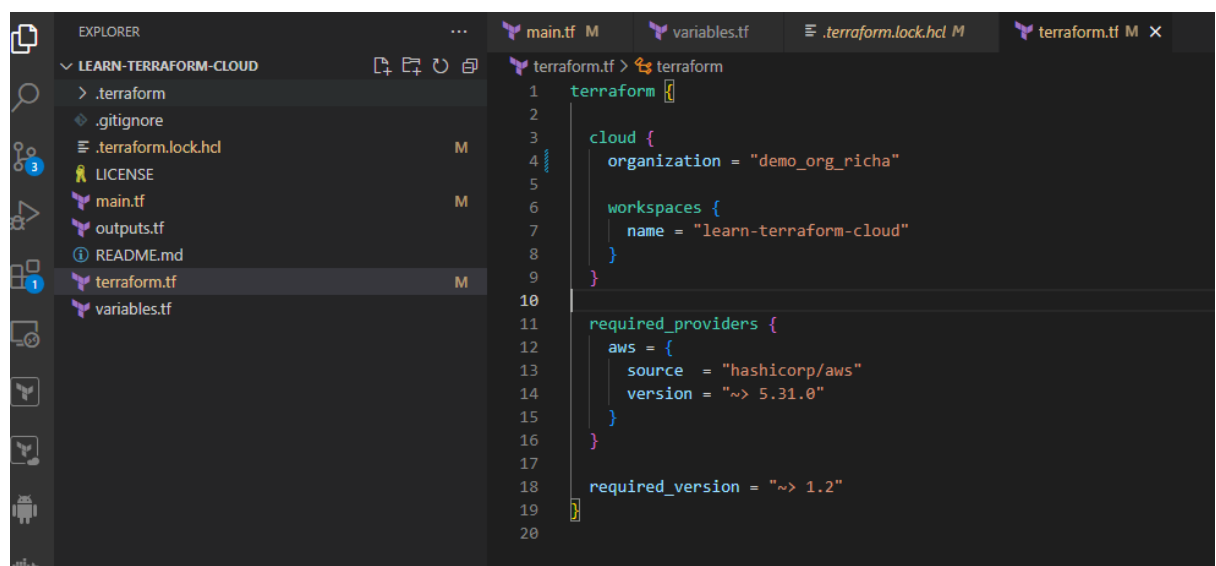
```
$ cd learn-terraform
```

Open the configuration in your code editor to review it (in my case- VS code editor) . It contains Terraform configuration files, which define and expose information about your infrastructure:

- **main.tf** specifies the AWS provider, a provider plugin that Terraform will use to manage your AWS resources. The configuration defines an EC2 instance and uses a Terraform data source to access a machine image to use for the instance.
- **variables.tf** defines the input variables that are referenced in main.tf, which make the configuration customizable. You will define these variables in the HCP Terraform UI later on in this tutorial.
- **outputs.tf** defines the information about your infrastructure that HCP Terraform will display to you when it makes changes.
- **terraform.tf** defines version constraints for Terraform and the AWS provider and the cloud block for the HCP Terraform integration.



Open the terraform.tf file and update the name of the organization in the cloud block to your own HCP Terraform organization.



Create the workspace

This cloud block specifies which HCP Terraform organization and workspace to use for the operations in this working directory. When using the CLI-driven HCP Terraform workflow, running terraform init on configuration with a cloud block creates the HCP Terraform workspace specified in the block, if it does not already exist.

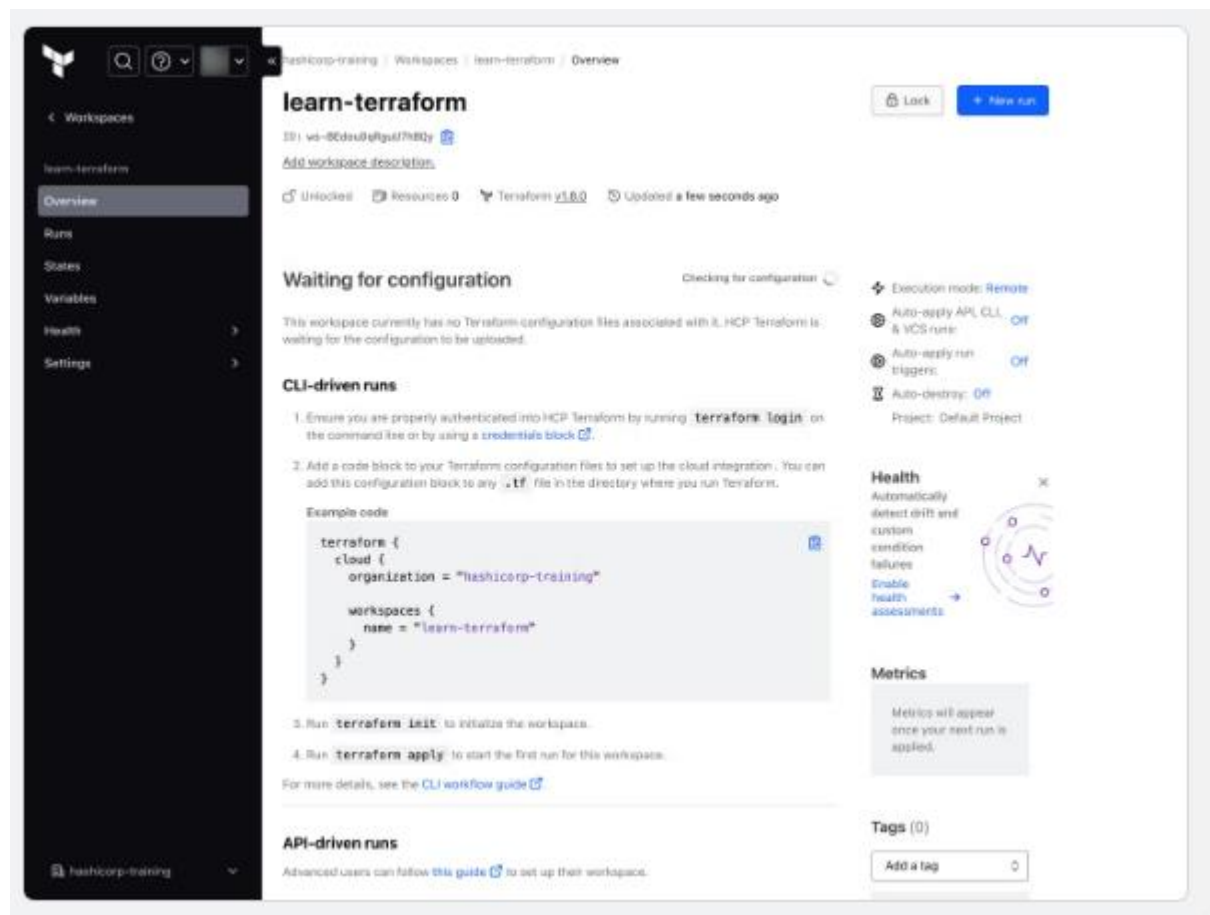
In your terminal, run terraform init to initialize the configuration and create your new HCP Terraform workspace.

note : Make sure that you have authenticated with HCP Terraform using the terraform login command prior to running this step.

then do,

terraform init

As part of the initialization process, Terraform created the new learn-terraform workspace in your HCP Terraform organization, configured for CLI-driven runs.

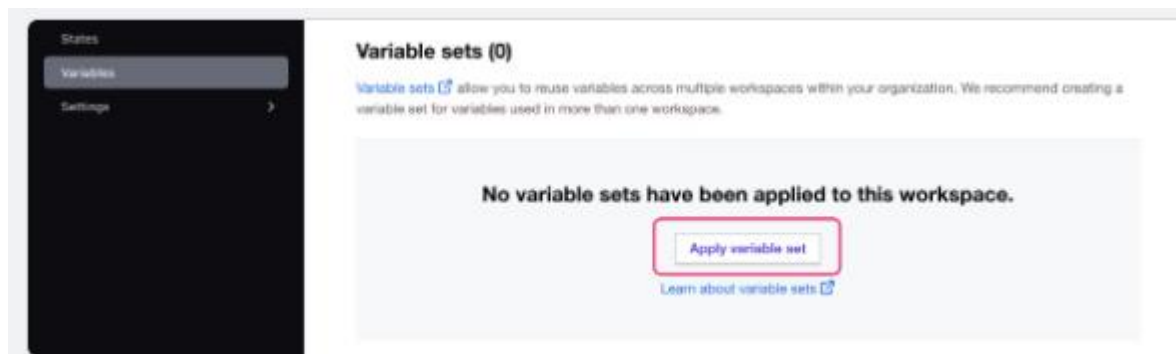


Every HCP Terraform workspace belongs to a project, which is a group of workspaces. When you create your workspace using this automated CLI workflow, HCP Terraform adds the workspace to your organization's **Default** project unless you set the project argument in your configuration's cloud block. Projects help you organize your workspaces into groups, making it easier to find workspaces in large HCP Terraform organizations.

Assign variable set to workspace

If you are not using a global variable set with your AWS credentials in your HCP Terraform organization, assign the variable set to your new workspace.

Navigate to your new learn-terraform workspace. Select Variables in the workspace's menu. Under Variable sets, click Apply variable set.



Select your AWS Credentials variable set, then click Apply variable set.



Your workspace can now use your AWS credentials. If you update the values of the variables in the variable set, those changes will propagate to this workspace and apply to all future runs.

Practical Lesson 10.5: Create an Infrastructure

You now have an HCP Terraform workspace configured to use AWS credentials defined in an HCP Terraform variable set. You can further configure your workspace using workspace-specific variables.

In this tutorial, you will customize your workspace by defining Terraform variables for your EC2 settings. You will then use HCP Terraform to provision your EC2 instance.

- **Configure Terraform variables**

Return to the HCP Terraform UI and navigate to the **Variables** page for your learn-terraform workspace. Find the **Workspace variables** section to set two variables that Terraform will use to configure your instance. Then, click **+ Add variable**.

First, define a Terraform variable by selecting the **Terraform variable** radio button. Set the key to `instance_type` and value to `t2.micro`.

Workspace variables (0)

Variables defined within a workspace always overwrite variables from variable sets that have the same type and the same key. Learn more about variable set [precedence](#).

Key	Value	Category
There are no variables added.		

Select variable category

☒ **Terraform variable**
These variables should match the declarations in your configuration. Click the HCL box to use interpolation or set a non-string value.

☐ **Environment variable**
These variables are available in the Terraform runtime environment.

Key **Value**

☐ HCL ⓘ ☐ Sensitive ⓘ

Description (Optional)

Click Add variable.

Then, click + Add variable again. Create another Terraform variable with the key `instance_name` and the value `Provisioned by Terraform`. Click Add variable.

Your workspace is now configured with 2 workspace-specific input variables and uses the AWS Credentials variable set as configured in the previous tutorial.

Workspace variables (2)

Variables defined within a workspace always overwrite variables from variable sets that have the same type and the same key. Learn more about variable set [precedence](#).

Key	Value	Category	
instance_name	Provisioned by Terraform	terraform	...
instance_type	t2.micro	terraform	...

+ Add variable

- **Apply planned changes**

Now that you have defined your configuration's input variables in your HCP Terraform workspace, you can provision your infrastructure.

You already initialized your configuration in the previous tutorial to create your new workspace. Now, run **terraform apply** to create your EC2 instance.

Terraform will trigger your run in HCP Terraform and stream the output to your terminal. Alternatively, you can follow and manage the run in the HCP Terraform UI.

```
PS C:\Users\Admin\learn-terraform-cloud> terraform plan
Running plan in Terraform Cloud. Output will stream here. Pressing Ctrl-C
will stop streaming the logs, but will not stop the plan running remotely.

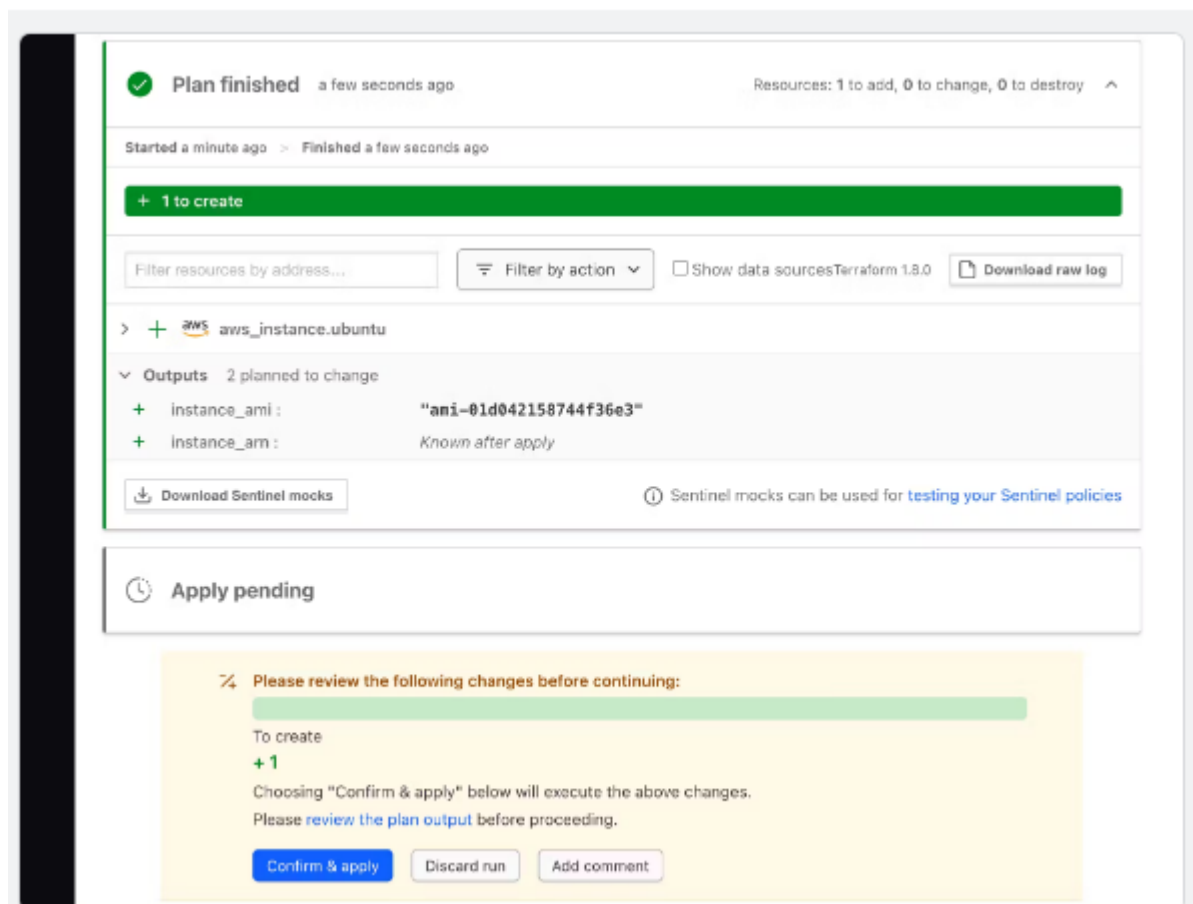
Preparing the remote plan...

To view this run in a browser, visit:
https://app.terraform.io/app/demo_org_richa/learn-terraform-cloud/runs/run-YsyZG2LksATHsxHB

Waiting for the plan to start...

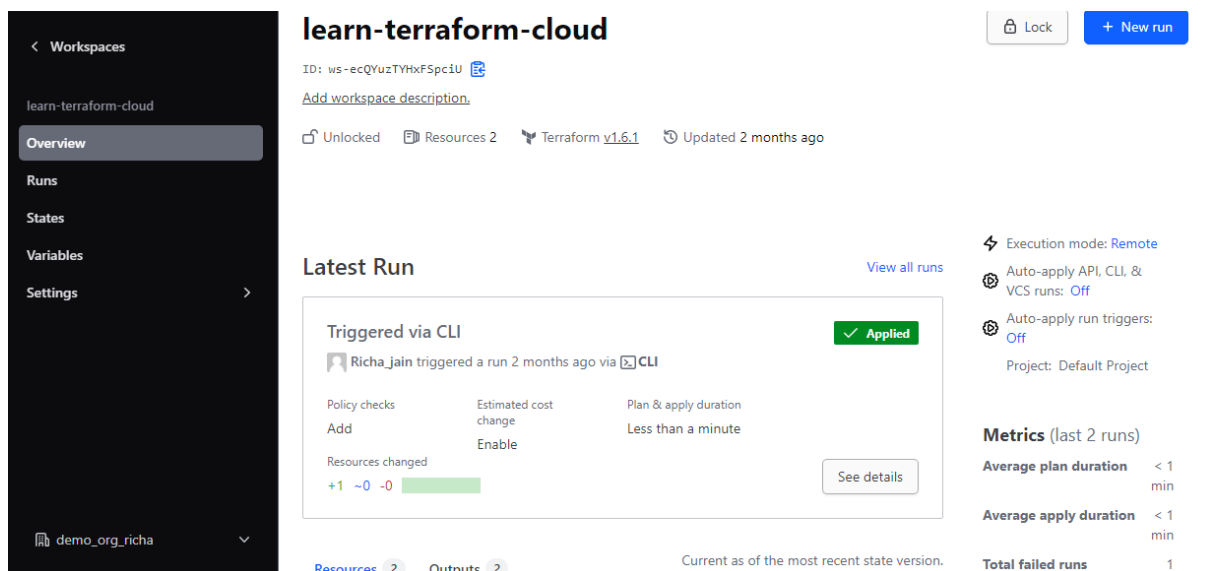
Terraform v1.6.1
on linux_amd64
Initializing plugins and modules...
data.aws_ami.ubuntu: Refreshing...
data.aws_ami.ubuntu: Refresh complete after 1s [id=ami-0b7dd63c6a810b1dd]
```

Navigate to the run URL that Terraform displays in your command output.

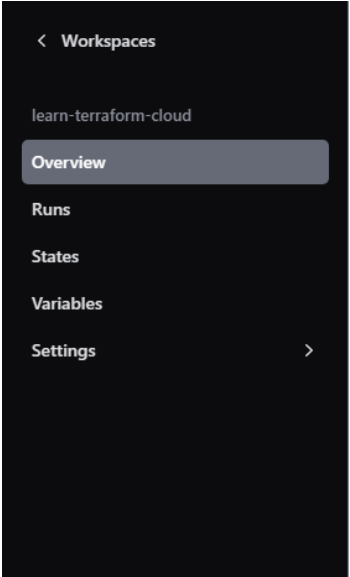


Once the plan is complete, click the Confirm & Apply button, followed by the Confirm Plan button to apply the change.

After a few minutes, your apply should complete. HCP Terraform will display a confirmation.



Now return to the workspace's Overview page. Under the details about your latest run, Terraform displays a table of the resources currently managed in the workspace.



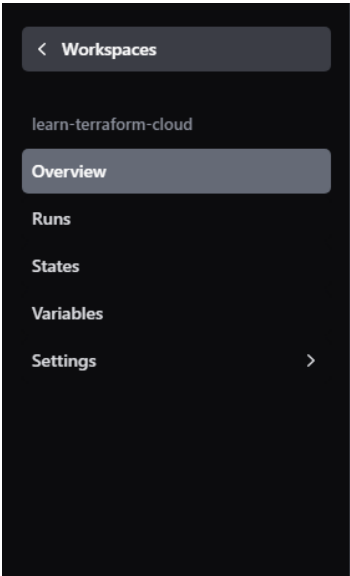
Resources 2 Outputs 2 Current as of the most recent state version.

Filter resources

NAME	PROVIDER	TYPE	MODULE	CREATED ↓
ubuntu	hashicorp/aws	data.aws_ami	root	Apr 10 2024
ubuntu	hashicorp/aws	aws_instance	root	Apr 10 2024

1 - 2 of 2 resources.

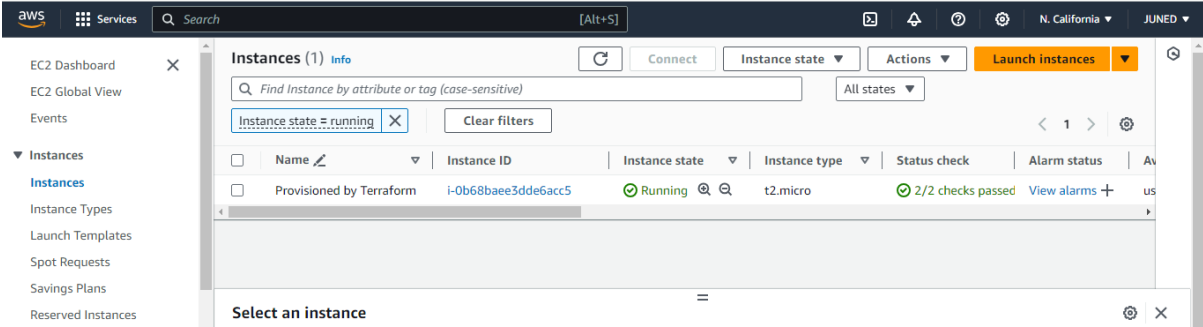
It also shows the workspace's outputs in the Outputs tab for easy access.



Resources 2 Outputs 2 Current as of the most recent state version.

NAME ↓	TYPE	VALUE
instance_ami	string	"ami-01d042158744f36e3"
instance_arn	string	"arn:aws:ec2:us-west-1:891377357490:instance/i-0b039c0256ab1eb19"

In this tutorial, you created an AWS EC2 instance using HCP Terraform. Verify that the infrastructure exists by visiting the AWS web console page for EC2. note: make sure to select your required Region.



END OF THE DOCUMENT